Software Design Document

MyHomeLib – Personal Library Cataloging App

Version 1.0

Author:
Grace Hechavarria

Peer Review Partner:
Yousef Habouh

Professor: Dr. Kiavash Bahreini

CEN 5064 Software Design – Fall 2026

Florida International University

October 10, 2026

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Grace Hechavarria	Initial release: architecture, components, detailed design, sprint planning, design rationale, and verification of the midterm build.	October 10, 2026

Contents

1 Introduction **3**
- 1.1 Purpose . . . 3
- 1.2 Customer, Setting, Sponsoring Organization, and Constraints . . . 3
- 1.3 Problem, Background, and Stakeholders . . . 4
- 1.4 Application Overview: Characteristics, Features, Differentiation, and Impact 4
- 1.5 Scope . . . 5
- 1.6 Definitions, Acronyms and Abbreviations . . . 5
- 1.7 Acknowledgements . . . 6
- 1.8 References . . . 6

2 System Overview **6**

3 System Components **7**
- 3.1 Decomposition Description . . . 7
- 3.2 Dependency Description . . . 9
- 3.3 Interface Description . . . 9
  - 3.3.1 Presentation View Model . . . 9
  - 3.3.2 Service Use Cases . . . 10
  - 3.3.3 Domain Model . . . 11
  - 3.3.4 Book Repositories . . . 12
  - 3.3.5 Book-Source Adapters . . . 12
  - 3.3.6 Device Adapters . . . 13
- 3.4 Module Interfaces . . . 13
- 3.5 User Interfaces (GUI) . . . 14
  - 3.5.1 To Be Read . . . 14
  - 3.5.2 Owned Catalog . . . 14
  - 3.5.3 Wanted . . . 15
  - 3.5.4 Scan . . . 16
  - 3.5.5 Book Detail . . . 16

4 Detailed Design **17**
- 4.1 Data Detailed Design . . . 19
- 4.2 RTM . . . 20

5 Sprint Planning **21**
- 5.1 Role Focuses . . . 22
- 5.2 Sprint Schedule, User Stories, and Acceptance Criteria . . . 22
  - 5.2.1 Proposed Timeline . . . 22
  - 5.2.2 Actual Timeline . . . 25

6 Design **28**
6.1 Definition of Design 28
6.2 Design Process 29
6.3 Graphical Representation of the Design 29
6.3.1 Use Case Diagram 29
6.3.2 Activity Diagram 30
6.3.3 Class Diagram 32
6.3.4 Sequence Diagram 34
6.4 Rationale Management 34
6.4.1 Issues Addressed 35
6.4.2 Alternatives Considered 35
6.4.3 Decisions Made 35
6.4.4 Criteria Used to Guide Decisions 36
6.4.5 Design Debate and Final Resolution 36

7 Verification **36**
7.1 Sprint 1 – Project Proposal and Approval 37
7.2 Sprint 2 – Tier Breakdown 37
7.3 Sprint 3 – C4 and UML Models 38
7.4 Sprint 4 – Scanning, Shelves, and Copies 38
7.5 Sprint 5 – Acquisition Rules 40
7.6 Sprint 6 – Exact Edition, Refactor, and Design System 41

8 References **44**

1 Introduction

1.1 Purpose

This Software Design Document (SDD) describes the architecture and system design of MyHomeLib, a cross-platform mobile application for cataloging a personal home library. It is intended for the course instructor, the peer review partner, and any software engineer who will maintain or extend the system. It explains how the application is divided into tiers and components, how those components communicate, why the main design decisions were made, and how each requirement was verified.

1.2 Customer, Setting, Sponsoring Organization, and Constraints

The primary customer is the end user: a bibliophile who owns a large or growing collection of physical books and wants to know exactly what is on their shelves, what they still want to buy, and what they have not read yet. The formal setting is the semester project for CEN 5064 Software Design at Florida International University. There is no external sponsor; the project is overseen by the course instructor, Dr. Kiavash Bahreini, and reviewed at two structured peer review sessions (midterm and final).

The course and instructor set the following constraints, which shaped the design:

- **Platform a partner can run.** The review partner must be able to clone the repository and run the app without a Mac, Xcode, Android Studio, or a paid developer account. This led to Expo Go (ADR-001, ADR-005).
- **One data store.** The project may use a single persistence technology. On a phone that is SQLite; on the web fallback the same port is backed by browser storage (ADR-002).
- **Limited external integrations, and no scraping.** Bookstore web scraping was proposed early and dropped at the instructor's request (ADR-004). Book metadata comes only from official APIs: Open Library (no key) and, optionally, Google Books (free key) (ADR-003, ADR-006).
- **Documented architecture.** The repository must show a tier breakdown, C4 diagrams (Context, Container, Component), UML class and sequence diagrams, and Architecture Decision Records, and the code must follow them.
- **Code quality.** The code must use classes to bundle data with behavior, apply the object-oriented techniques that fit the problem, avoid duplication, and keep every stylesheet in its own imported file.
- **Semester timeline.** Work is planned in weekly sprints from August to November 2026.

1.3 Problem, Background, and Stakeholders

Collectors with hundreds of books regularly buy a second copy of something they already own, lose track of which edition they have, and forget which books on their shelves are still unread. Gifts are another blind spot: months later it is hard to remember who gave a book. General-purpose reading apps focus on reviews and social feeds; they do not model a physical collection, its editions, its duplicates, or how each copy was acquired.

The stakeholders are:

- **The bibliophile (primary user)**, who scans, shelves, and browses their own books.
- **The course instructor**, who evaluates the architecture, the code quality, and this document.
- **The peer review partner**, who must be able to run the app from the README and compare the diagrams to the code.
- **External data providers** (Open Library and Google Books), whose public APIs the app depends on and whose terms it must respect.

1.4 Application Overview: Characteristics, Features, Differentiation, and Impact

The application is characterized by: (1) **cross-platform without native builds**, a single React Native (Expo) codebase that runs in the free Expo Go client on iPhone and Android, with Expo web as a conference fallback; (2) **exact-edition scanning**, where the ISBN barcode is read with the camera and the edition's own publisher, date, page count, binding, and cover are looked up, with the owner choosing the cover that matches their copy or photographing it; and (3) **collection rules in the domain model**, so that the app, not the user, guarantees that the catalog stays consistent.

The core features are:

1. **ISBN scan** with the camera, or a typed ISBN when no camera is available.
2. **Owned and Wanted shelves**. "To be read" (TBR) is a view of Owned books split into Unread and Read, not a third shelf.
3. **Owned catalog** grouped A–Z, by author, or by series, with an A–Z letter rail for fast scrolling.
4. **Domain rules**: scanning an ISBN that is already cataloged adds a copy instead of a second entry; a book cannot be Owned unless it was purchased or gifted; a gift requires the giver's name; reading status is independent of the shelf.
5. **Book detail** with edition facts, binding, read/unread toggle, shelf moves, a cover photo option, and deletion with confirmation.

Unlike reading-tracker apps, MyHomeLib records the physical copy: which edition and binding it is, how many copies the owner has, and whether each one was bought or received as a gift. It runs entirely on the user's device with no account, so the library stays private. The expected impact is fewer duplicate purchases, a reliable to-be-read list, and a record of gifts.

1.5 Scope

This document covers the midterm version of MyHomeLib. The system consists of four tiers (Presentation, Service, Domain, Data) and six major components: the Presentation view model, the Service use cases, the Domain model, the book repositories, the book-source adapters, and the device adapters (camera and haptics). Each component is described in Section 3. Accounts, cloud synchronization, price tracking, and bookstore integrations are out of scope for this version.

1.6 Definitions, Acronyms and Abbreviations

Term	**Meaning**
ADR	Architecture Decision Record: a one-page record of a design decision, its context, and its consequences
API	Application Programming Interface
C4	Context, Container, Component, Code: a model for diagramming software architecture at four zoom levels
Expo Go	Free client app that runs an Expo (React Native) project on a phone without a native build
ISBN	International Standard Book Number; ISBN-13 barcodes are EAN-13
MVVM	Model–View–ViewModel: a presentation pattern in which an observable view model holds screen state and logic
OOP	Object-Oriented Programming
Port	An interface owned by the Service tier that the Data tier implements
SDD	Software Design Document
TBR	"To be read": Owned books, split into Unread and Read
UML	Unified Modeling Language
WCAG	Web Content Accessibility Guidelines

1.7 Acknowledgements

The requirements, domain rules, feature decisions, and design direction were set by Grace Hechavarria, who reviewed and approved every change. Anthropic's Claude (Opus 5.5), used through the Claude Code tool (Anthropic, 2026), assisted with implementation, refactoring toward the class-based architecture, automated tests, rendering the Mermaid diagrams to images, capturing the application screenshots, and drafting and assembling this document. Design-review skill packages (Anthropic frontend-design, TasteSkill, UI/UX Pro Max, Vercel React Native guidelines, AccessLint, and the Bencium UX designer guide) were used as checklists during the interface redesign. The document was typeset with LaTeX (MiKTeX). The structure of this SDD follows a template from a previous FIU software engineering course. Thanks to the peer review partner, Yousef Habouh, for the structured reviews.

1.8 References

- Anthropic. (2026). *Claude* (Opus 5.5 model) [Large language model], accessed through *Claude Code* [Computer software]. Retrieved from https://www.anthropic.com/claude-code
- Expo. (n.d.). *Expo documentation*. Retrieved from https://docs.expo.dev/
- Internet Archive. (n.d.). *Open Library APIs*. Retrieved from https://openlibrary.org/developers/api
- Google. (n.d.). *Books API*. Retrieved from https://developers.google.com/books
- Brown, S. (n.d.). *The C4 model for visualising software architecture*. Retrieved from https://c4model.com/

2 System Overview

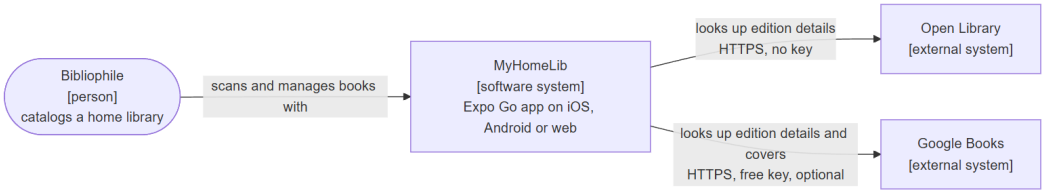


Figure 1: C4 Level 1 – System Context

Figure 1 shows MyHomeLib in its environment. A single type of user, the bibliophile, catalogs books with the app. The app depends on two external systems for book metadata: Open Library, which needs no key, and Google Books, which is optional and needs a free API key. Everything else runs on the user's device.

Figure 2 shows the architecture chosen for MyHomeLib: a **four-tier modular monolith** in one TypeScript codebase. The tiers are folders with a strict dependency direction, enforced by automated tests:

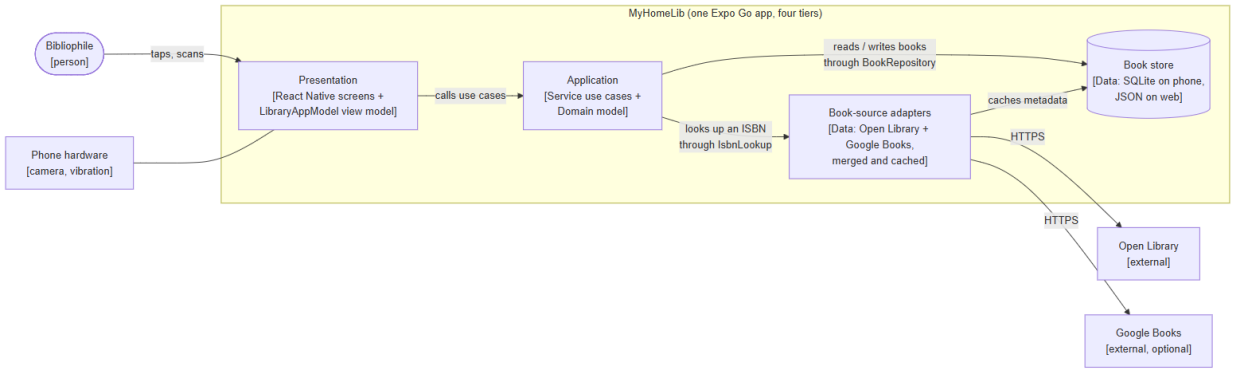


Figure 2: C4 Level 2 – Container (System Architecture)

- **Presentation** (`presentation/`): React Native screens rendered by Expo, and the `LibraryAppModel` view model that holds all screen state and user flows. Components only render and forward taps.
- **Service** (`service/`): one class per use case (scan a book, move a book, set reading status, and so on) plus the *ports*, the interfaces that the Data tier implements.
- **Domain** (`domain/`): the business objects and rules: `Book`, `PersonalLibrary`, and the acquisition records `Purchase` and `Gift`. Domain imports nothing outside itself.
- **Data** (`data/`): the adapters to the outside world: SQLite on a phone, browser storage on the web, and the Open Library and Google Books lookups, merged and cached.

Calls always flow Presentation → Service → Domain and ports, never backwards. The Data tier depends on the Service ports, not the other way around, so the business rules can be tested with in-memory fakes and the storage technology can change without touching them. A composition root (`bootstrap.ts`) is the only place that chooses concrete adapters and wires them into the use cases.

Because MyHomeLib runs in Expo Go, the same code reaches iOS, Android, and the web. The architecture can be extended with cloud synchronization or a second user by adding a new repository adapter behind the same port; this is out of scope for this version.

3 System Components

3.1 Decomposition Description

Figure 3 shows a top-down decomposition of the codebase. Each tier is a folder, and each folder holds classes with one responsibility each:

The **Presentation view model** (`presentation/model/`) is the controller of the C4 component view. `LibraryAppModel` extends a reusable `Observable` base class and holds every screen's state: the selected tab, the open popup, the scan draft, and the flash message. `LibraryController` turns screen intents into use case calls and returns a `LibraryView` snapshot. Helper classes keep logic out of the views: `ScanDraft` (a scan in progress), `Catalog`

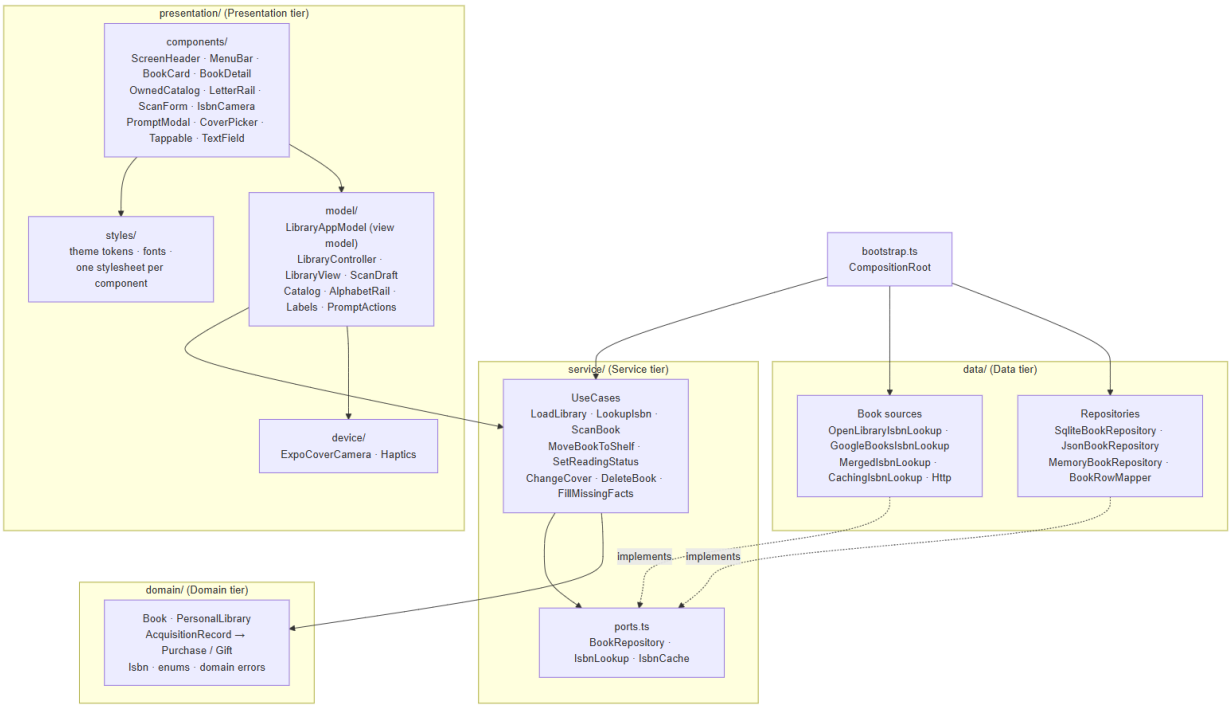


Figure 3: System Decomposition Diagram (folders and their classes)

and its grouping strategies, `AlphabetRail` (letter-rail math), `Labels` (all display wording), and `PromptActions` (popup buttons).

The **Presentation views** (`presentation/components/` and `presentation/styles/`) are React components that only render. Every component imports its stylesheet from `styles/`, and every color, size, and font comes from the theme tokens. Shared building blocks (`Tappable`, `TextField`, `BookCover`, `CoverBackdrop`, `Logo`, `Reveal`) replace repeated code.

The **Device adapters** (`presentation/device/`) wrap phone hardware: `ExpoCoverCamera` implements the `CoverCamera` interface for cover photos, and `Haptics` gives physical feedback.

The **Service use cases** (`service/`) orchestrate one user goal each. Eight use case classes are built once by the `UseCases` class from two ports.

The **Domain model** (`domain/`) enforces the collection rules. Its state is private and changes only through methods that check the rules.

The **Book repositories** (`data/`) persist the library: `SqliteBookRepository` on a phone, `JsonBookRepository` on the web, and `MemoryBookRepository` for tests, all translating rows through `BookRowMapper`.

The **Book-source adapters** (`data/`) look up an ISBN: `OpenLibraryIsbnLookup` and `GoogleBooksIsbnLookup`, combined by `MergedIsbnLookup` and cached by `CachingIsbnLookup`. All HTTP goes through one `Http` class.

3.2 Dependency Description

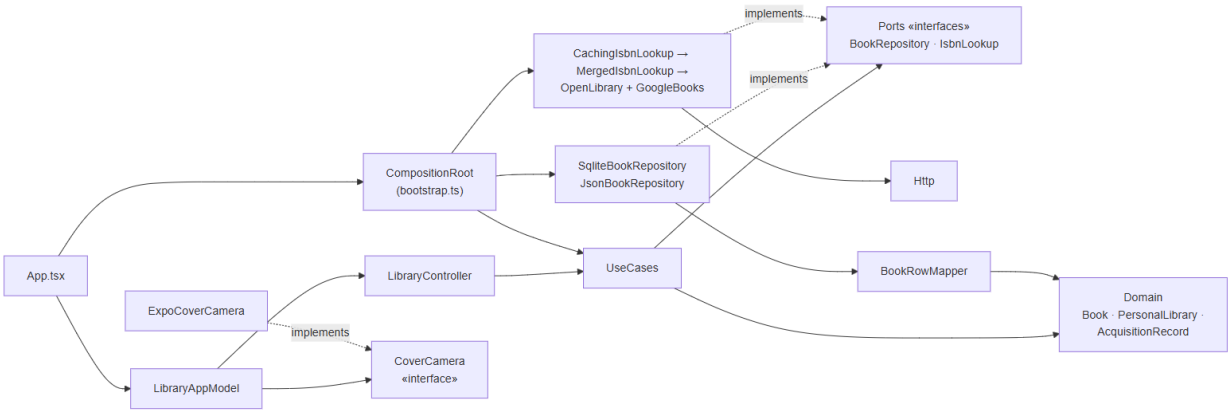


Figure 4: System Dependency Diagram

Figure 4 shows how each module depends on the others. `App.tsx` creates the `LibraryAppModel`, which uses `LibraryController` to reach the `UseCases`. The use cases depend on the Domain classes and on two *interfaces*, `BookRepository` and `IsbnLookup`. The concrete repositories and lookups depend on those interfaces (they implement them), which inverts the usual direction: Service never imports Data (the Dependency Inversion Principle, ADR-009).

`CompositionRoot` in `bootstrap.ts` is the only module that knows every concrete class. At start-up it opens SQLite (or browser storage when SQLite is unavailable), wraps the merged book sources in the cache, and hands the finished `UseCases` to the view model. The camera follows the same idea one tier up: the view model depends on the `CoverCamera` interface, and the app passes `ExpoCoverCamera` while tests pass a fake.

These rules are not only documented; `tests/architecture.test.ts` reads every source file and fails if Domain imports anything outside Domain, if Service imports Data or calls `fetch`, if Data imports Presentation or React, if Presentation reaches a repository, if a component defines its own stylesheet, or if any file other than `App.tsx` reaches the composition root.

3.3 Interface Description

3.3.1 Presentation View Model

Figure 5 shows the interface between the screens and the rest of the app. Screens read `LibraryAppModel`'s state and derived values (`heading`, `hint`, `page`, `bookList`) and call its intent methods (`scan`, `confirmScan`, `chooseGifted`, `moveSelected`, ...). React subscribes through one hook, `useObservable`, so every state change re-renders the screens. The model calls the use cases only through `LibraryController`, and takes cover photos only through the `CoverCamera` interface.

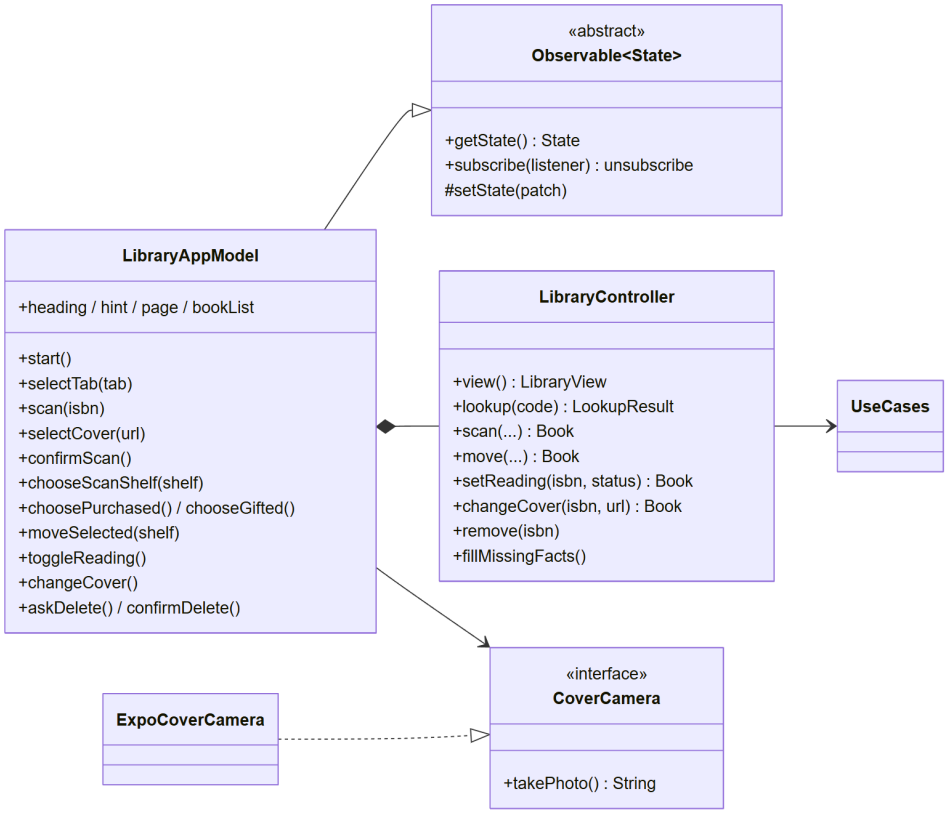


Figure 5: Presentation View Model Interface

3.3.2 Service Use Cases

Figure 6 shows the use case classes. Every library use case extends the abstract `LibraryUseCase`, which implements the shared load → change → save steps once (`updateBook`, a Template Method). Use cases that also need the book sources extend `LibraryLookupUseCase`. `LookupIsbnUseCase` only reads, so it stands alone. Each use case exposes a single `execute(...)` method.

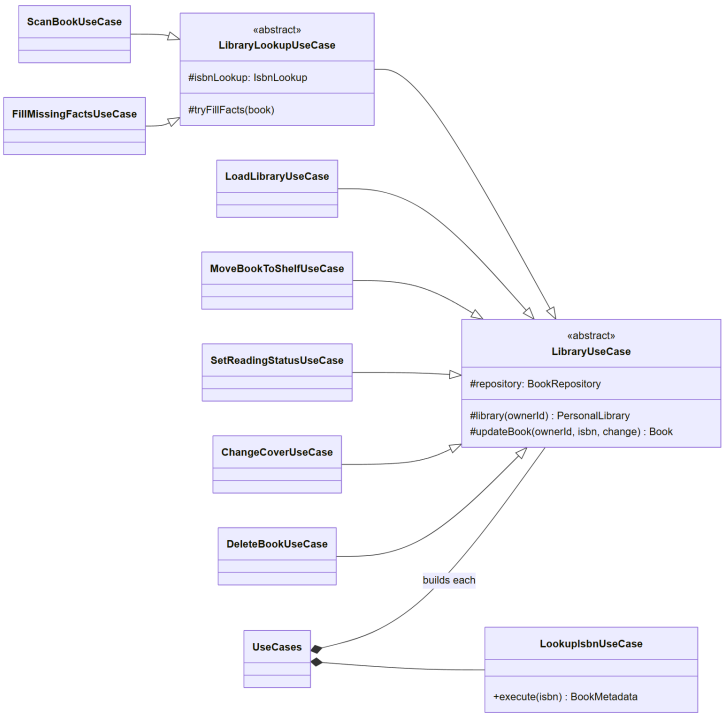


Figure 6: Service Use Case Interface

3.3.3 Domain Model

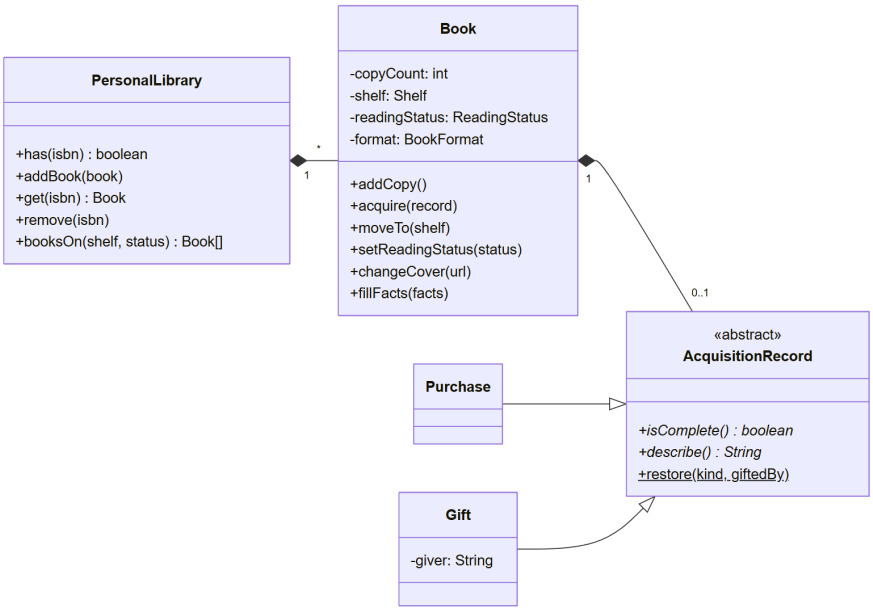


Figure 7: Domain Model Interface

Figure 7 shows the domain interface. `PersonalLibrary` composes the `Book` objects of one owner. `Book` keeps its fields private and offers intention-revealing methods (`addCopy`,

`acquire`, `moveTo`, . . .) that throw a domain error when a rule is broken. `AcquisitionRecord` is abstract; `Purchase` and `Gift` implement `isComplete()` and `describe()` polymorphically, so `Book` never branches on the acquisition type.

3.3.4 Book Repositories

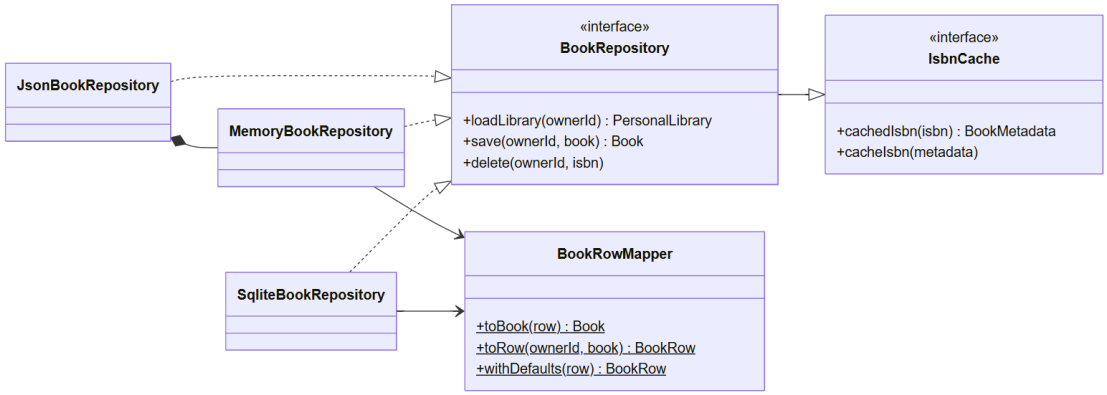


Figure 8: Book Repository Interface

Figure 8 shows the persistence interface. `BookRepository` extends `IsbnCache`, so one store holds both the library and the cached lookups. Three adapters implement it. `JsonBookRepository` reuses `MemoryBookRepository` by composition and only adds loading and saving to browser storage. `BookRowMapper` is the single translation between a stored row and a `Book` (Data Mapper pattern), including the defaults for rows saved by older versions.

3.3.5 Book-Source Adapters

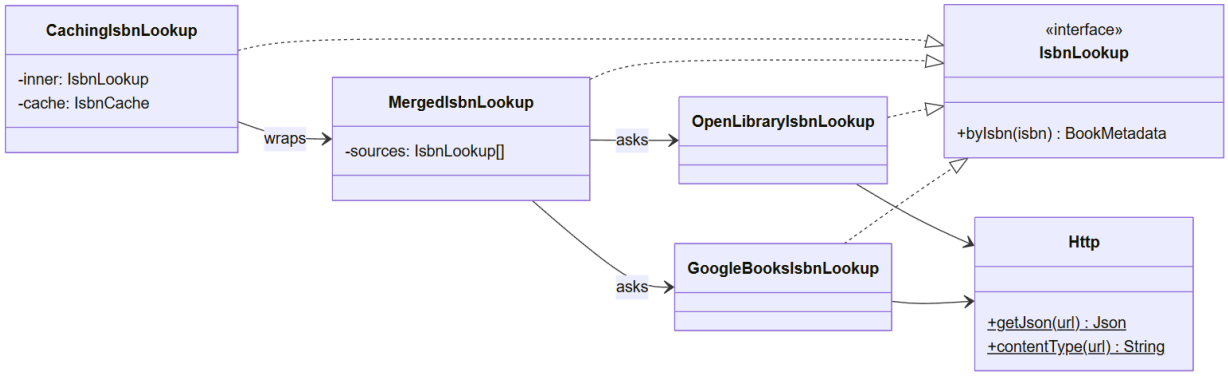


Figure 9: Book-Source Adapter Interface

Figure 9 shows the lookup chain. Every element implements the same `IsbnLookup` interface, so they stack as decorators: `CachingIsbnLookup` answers from the cache or asks `MergedIsbnLookup`, which asks Google Books and Open Library together, merges the answers field by field (edition data first), and keeps every distinct cover as a candidate. Both sources reach the network only through the `Http` class.

3.3.6 Device Adapters

The camera and the vibration motor are wrapped so that the view model never touches device APIs directly. `ExpoCoverCamera` implements `CoverCamera`: it opens the camera, lets the owner crop to the cover, and shrinks the photo to a small JPEG stored with the book. `Haptics` gives a light tick when a tab changes, a soft tap for a message, and a buzz for an error, and does nothing on the web. The ISBN barcode scanner is `IsbnCamera`, a view that reports each new code to the view model.

3.4 Module Interfaces

The following TypeScript interfaces are the contracts between tiers. The Service tier owns them (`service/ports.ts`); the Data tier implements them.

```
/** What the book sources know about one ISBN (one edition). */
export type BookMetadata = BookFacts & {
  isbn: string; title: string; author: string; publisher: string;
  coverUrl: string; // best cover first
  coverCandidates: string[]; // every distinct cover the sources offer
};

export interface IsbnLookup {
  byIsbn(isbn: string): Promise<BookMetadata>;
}

export interface IsbnCache {
  cachedIsbn(isbn: string): Promise<BookMetadata | null>;
  cacheIsbn(metadata: BookMetadata): Promise<void>;
}

export interface BookRepository extends IsbnCache {
  loadLibrary(ownerId: string): Promise<PersonalLibrary>;
  save(ownerId: string, book: Book): Promise<Book>;
  delete(ownerId: string, isbn: string): Promise<void>;
}
```

The Presentation tier owns two more contracts: the device camera, and the reusable observable base class of the view model.

```
export interface CoverCamera {
  // A small image of the cover (data URI), or null if the owner backed out.
  takePhoto(): Promise<string | null>;
}

export abstract class Observable<State> {
  getState(): State;
  subscribe(listener: () => void): () => void; // returns unsubscribe
  protected setState(patch: Partial<State>): void;
}
```

3.5 User Interfaces (GUI)

The interface uses one design system: a dark "enchanted forest" background, a single mint accent reserved for selection and primary actions, the Outfit typeface in four weights, and a floating tab dock with TBR, Owned, Wanted, and Scan. All screenshots below were captured from the running application at phone size.

3.5.1 To Be Read

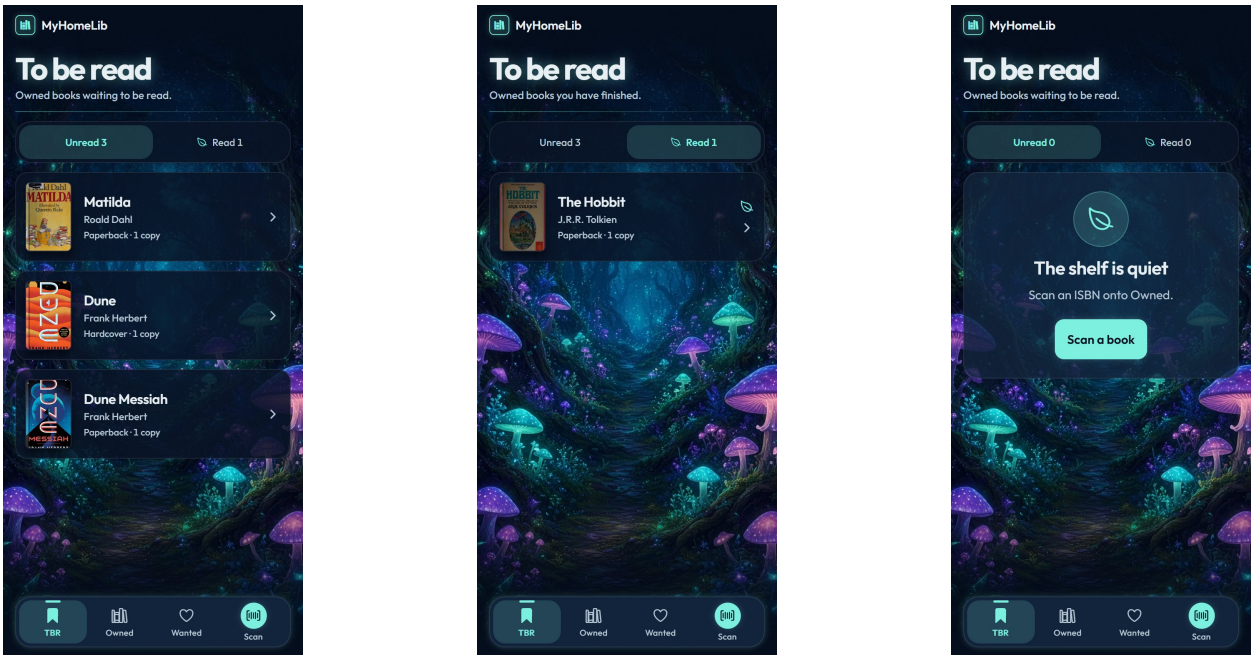


Figure 10: To be read: Unread (left), Read (center), and the empty state (right)

Figure 10 shows the TBR screen, the app's home. The switch at the top splits Owned books into Unread and Read, with a count on each side. Each card shows the cover, title, author, binding, and copy count; a leaf marks books already read. An empty shelf explains itself and offers a Scan button.

3.5.2 Owned Catalog

Figure 11 shows the full Owned catalog. The switch chooses the grouping; each section starts with an index heading. In the series grouping, a series appears as a fan of its covers with a count badge, and tapping it expands its books. The A–Z rail on the right jumps to a section when tapped or dragged.

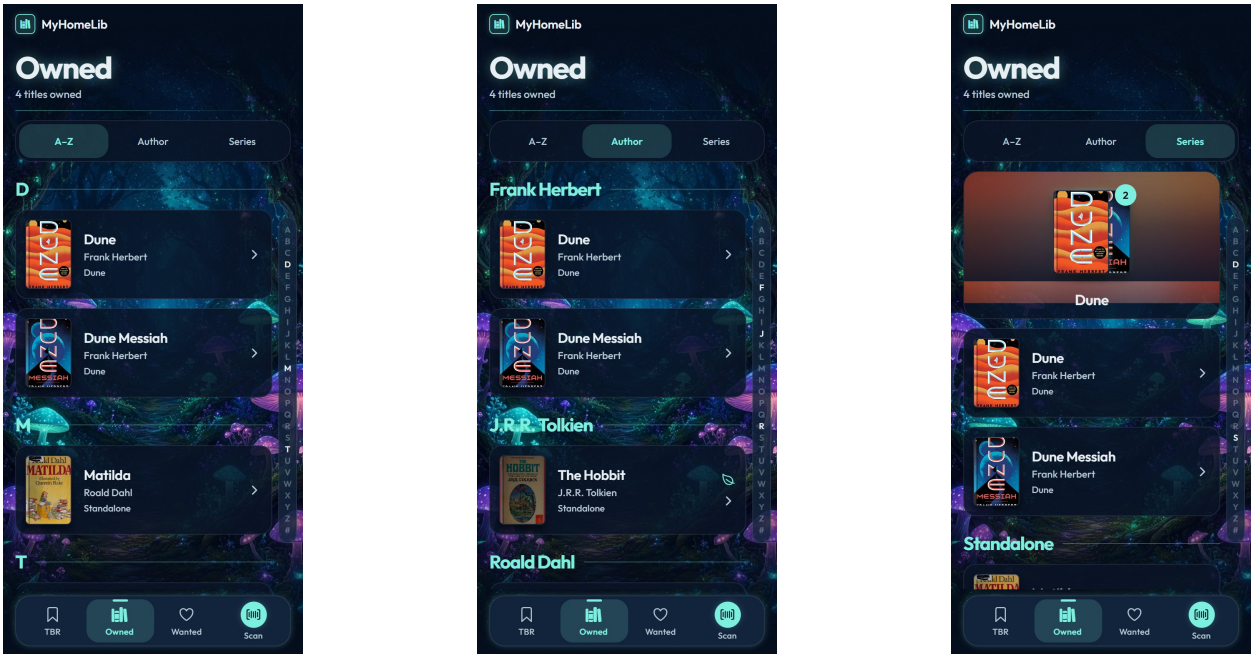


Figure 11: Owned catalog grouped A–Z (left), by author (center), and by series (right)

3.5.3 Wanted

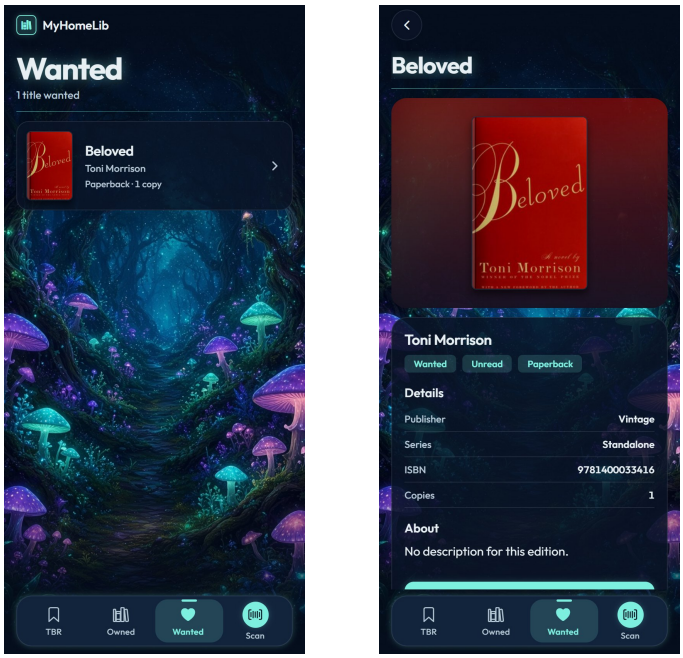


Figure 12: Wanted list (left) and a Wanted book's detail (right)

Figure 12 shows the wish list. A Wanted book has the same detail screen as an Owned one, with a "To Owned" action instead of "To Wanted".

3.5.4 Scan

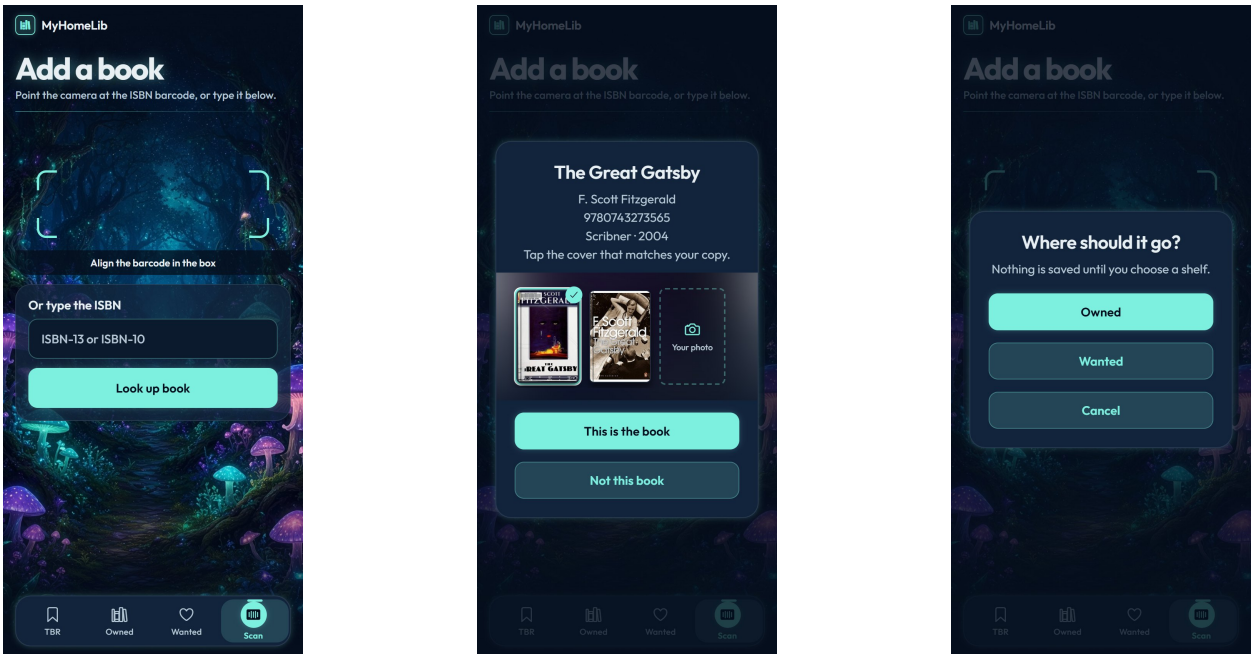


Figure 13: Scan screen (left), the confirm popup with the cover picker (center), and the shelf choice (right)

Figure 13 shows the scan flow. The camera reads the barcode inside the corner brackets; the ISBN can also be typed. The confirm popup shows the edition's details and every cover the sources offer, on a soft halo of the selected cover, plus a "Your photo" tile. Nothing is saved until the owner has answered every question.

3.5.5 Book Detail

Figure 14 shows a book's detail screen: the cover on a halo of its own colors, status tags, the edition's facts (publisher, date, pages, series, ISBN, copies, and how it was acquired), the description and subjects, and the actions: mark read or unread, move between shelves, replace the cover with a photo, and delete.

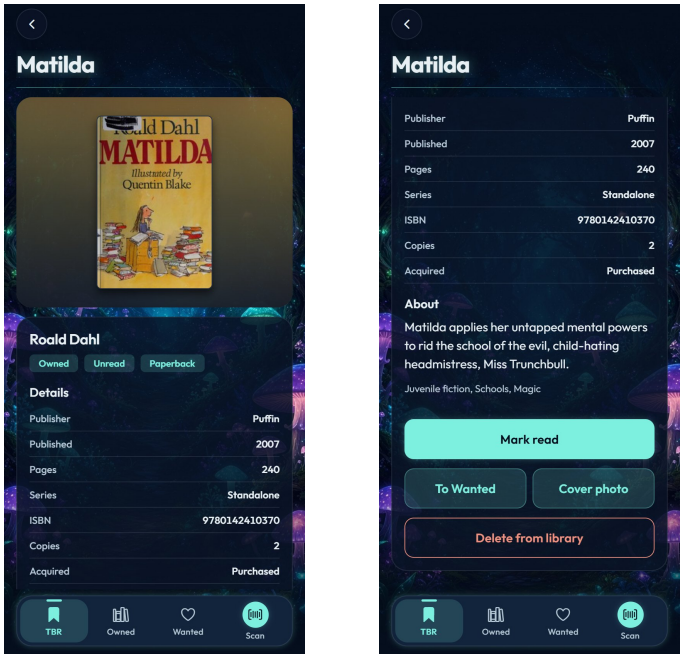


Figure 14: Book detail: cover and facts (left), description and actions (right)

4 Detailed Design

Sequence Diagrams

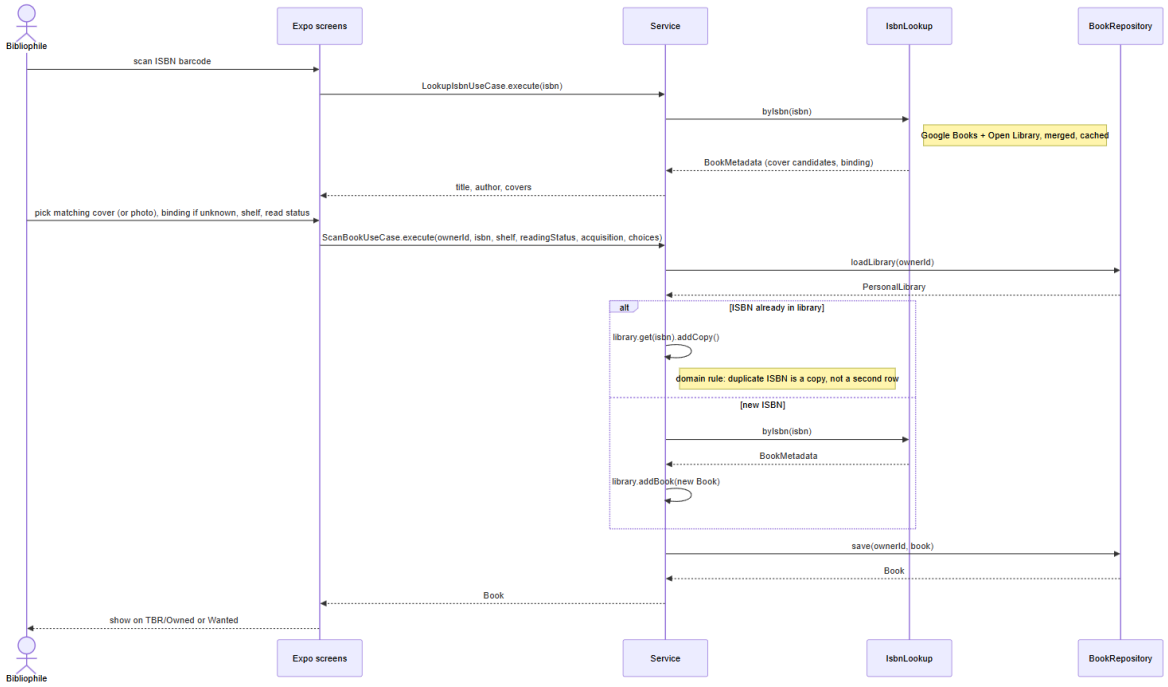


Figure 15: Sequence diagram: scanning a book (lookup, confirm, save)

Figure 15 shows the main use case. The lookup saves nothing; it returns the edition's

details and cover candidates. Only after the owner confirms does `ScanBookUseCase` load the library, add a copy or a new `Book`, and save it.

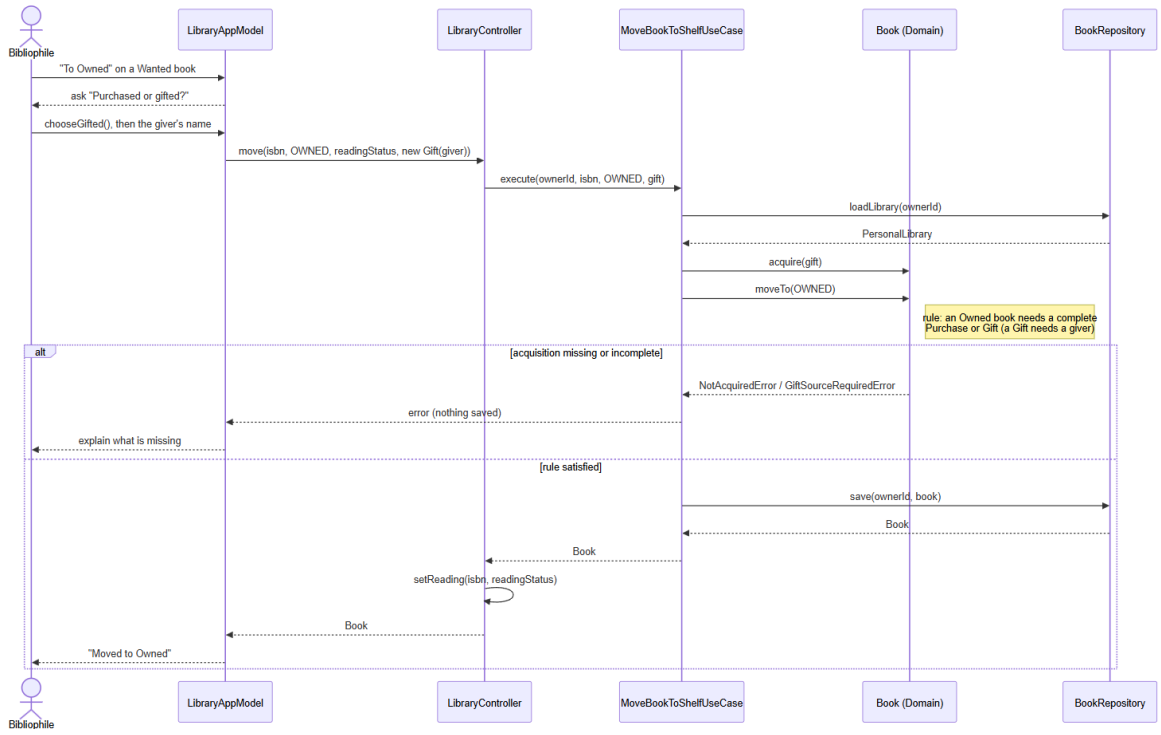


Figure 16: Sequence diagram: moving a Wanted book to Owned as a gift

Figure 16 shows how a domain rule is enforced. The view model gathers the acquisition first; `Book.moveTo` then refuses to move a book to Owned without a complete `Purchase` or `Gift`, and nothing is saved when the rule fails.

Pseudocode (scan a book)

```
User opens Scan
If camera permission granted:
    Read the ISBN barcode from the camera preview
Else:
    User types the ISBN
metadata = cache.get(isbn) or merge(GoogleBooks(isbn), OpenLibrary(isbn))
If no source knows the ISBN:
    Show "No book found for ISBN ..."; stop
Show the edition and its cover candidates
If the ISBN is already in the library:
    On confirm: book.addCopy(); save(book); stop
Owner picks the cover that matches their copy (or photographs it)
If the binding is unknown:
    Ask "Hardcover or paperback?"
Ask "Owned or Wanted?"
If Owned:
```

```
    Ask "Purchased or gifted?"
    If gifted: ask for the giver's name (required)
Ask "Read or unread?"
library.addBook(new Book(metadata, choices))   // domain rules checked here
save(book); cache(metadata)
Show the book on TBR / Owned or Wanted
```

4.1 Data Detailed Design

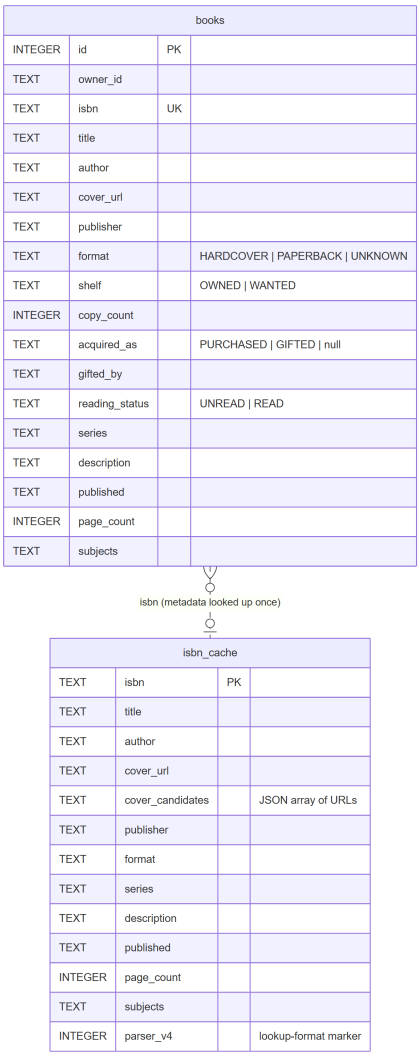


Figure 17: Local data model (SQLite)

Figure 17 shows the two tables the app stores on the device. On the web the same data is kept as JSON in browser storage with the same field names.

- **books**: one row per cataloged title and owner: the edition facts (title, author, publisher, date, pages, series, subjects, description), the chosen cover (a URL or an inline

photo), the binding, the shelf, the copy count, the reading status, and the acquisition (`PURCHASED` or `GIFTED` with the giver's name). The ISBN is unique, which backs the "duplicate ISBN adds a copy" rule.

- **isbn_cache**: one row per looked-up ISBN with the merged metadata and every cover candidate (as a JSON array), so a book is fetched from the network only once. A version marker clears old entries when the lookup format changes.

The schema migrates itself: missing columns are added when the app starts, and rows saved by older builds (for example the former third "TBR" shelf, now Owned and unread) are upgraded by `BookRowMapper`.

4.2 RTM

The Requirements Traceability Matrix links each requirement to the components that implement it and to the verification cases in Section 7.

Req.-ID	Requirement Description	Design Component	Test-Case #
FR-1	Scan an ISBN with the camera, or type it	IsbnCamera, ScanForm, LibraryAppModel	15, 16, 21
FR-2	Look up the exact edition from Open Library and Google Books	OpenLibraryIsbnLookup, GoogleBooksIsbnLookup, MergedIsbnLookup	16, 21, 22
FR-3	Pick the matching cover or photograph your own	CoverPicker, ScanDraft, CoverCamera	17, 23
FR-4	Ask for the binding only when no source knows it	ScanDraft, LibraryAppModel	18
FR-5	Shelve a scanned book as Owned or Wanted	ScanBookUseCase, PersonalLibrary	14, 19
FR-6	An Owned book must be purchased or gifted; a gift needs the giver's name	Book, AcquisitionRecord, Purchase, Gift	10–14
FR-7	A duplicate ISBN adds a copy instead of a second entry	Book.addCopy, ScanBookUseCase	8
FR-8	Reading status is independent of the shelf; TBR shows Owned unread/read	Book, LibraryView, SetReadingStatusUseCase	4, 5, 24
FR-9	Browse Owned A–Z, by author, or by series with a letter rail	Catalog, CatalogGrouping strategies, AlphabetRail	6, 7
FR-10	Move a book between shelves	MoveBookToShelfUseCase, Book.moveTo	10–13

Req.-ID	Requirement Description	Design Component	Test-Case #
FR-11	Delete a book only after confirmation	DeleteBookUseCase, LibraryAppModel	25
FR-12	Cancel any popup without changing the library	LibraryAppModel .cancelPrompt	20
FR-13	Persist the library and cache lookups on the device	SqliteBookRepository, JsonBookRepository, CachingIsbnLookup	9
NFR-1	Tiers depend only downward; Service never imports Data	Ports, CompositionRoot, architecture tests	26
NFR-2	Stylesheets are imports; values come from theme tokens	presentation/styles, architecture tests	26
NFR-3	Runs in Expo Go (iOS, Android) and on the web without native builds	Expo modules only (ADR-005, ADR-008)	27
NFR-4	Accessible: WCAG AA contrast, screen-reader labels, reduced motion	Theme tokens, Tappable, Reveal, Labels	28
NFR-5	The architecture is documented: tier table, C4 and UML diagrams, ADRs	README, docs/adr, this SDD	1–3

5 Sprint Planning

MyHomeLib is an individual project, so every development role below is held by the same developer, with a peer review partner reviewing the work at the midterm and final sessions (October 12 and 19, and November 9 and 16). Work is organized as weekly sprints that follow the course milestones. The *Proposed Timeline* below is the plan at project approval; the *Actual Timeline* records what was built. The sprint dates stayed in the same calendar windows, but the content of two sprints changed. In proposed Sprint 5, the Wanted list was to include a bookstore price check by scraping retailer pages; the instructor asked for scraping to be removed (ADR-004), so actual Sprint 5 concentrated on the acquisition rules for moving books to Owned. In proposed Sprint 6, the plan was polish and testing; testing showed that Open Library alone often returned the wrong cover for a specific printing, so actual Sprint 6 added exact-edition lookup with Google Books (ADR-006), and refactored the code into classes with a view model and a design system (ADR-007, ADR-008) to meet the course's object-oriented requirements. Sprints 7–9 have not started yet and follow the proposed timeline.

5.1 Role Focuses

Team Member	Role	Responsibilities
Grace Hechavarria	Architect, Developer, UI/UX Designer, Tester	Requirements and domain rules, architecture and ADRs, all four tiers, interface design, automated tests, and documentation
Yousef Habouh	Peer Review Partner	Runs the app from the README, compares the diagrams to the code, and files the structured midterm and final reviews

5.2 Sprint Schedule, User Stories, and Acceptance Criteria

5.2.1 Proposed Timeline

Sprint 1 – Project Proposal and Approval

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 1	08/24 – 08/30	As the instructor, I want a clear project idea and platform, so that I can approve the scope.	1. Verify that the repository exists with a README. 2. Verify that the approval paragraph names the platform and the core features.

Team Member	Task
Grace Hechavarria	Create the repository; draft project ideas; write the approval paragraph

Sprint 2 – Tier Breakdown

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 2	08/31 – 09/06	As a developer, I want each tier's responsibilities written down, so that every class has an obvious home.	1. Verify that the README has a Presentation / Service / Domain / Data table. 2. Verify that each row names the system's real responsibilities.

Team Member	Task
Grace Hechavarria	Write the tier table for the README

Sprint 3 – C4 and UML Models

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 3	09/07 – 09/20	As a reviewer, I want C4 and class diagrams, so that I can understand the system before reading code.	1. Verify that the README has C4 Context, Container, and Component diagrams. 2. Verify that a UML class diagram shows the domain rules.

Team Member	Task
Grace Hechavarria	Draw the C4 diagrams and the UML class diagram

Sprint 4 – Scanning, Shelves, and Copies

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 4	09/21 – 09/27	As a bibliophile, I want to scan an ISBN and shelve the book as Owned or Wanted, so that I can build my catalog quickly.	1. Verify that an ISBN can be scanned or typed. 2. Verify that a duplicate ISBN adds a copy. 3. Verify that TBR lists Owned books by reading status.

Team Member	Task
Grace Hechavarria	Build the four tiers, the scan flow, the domain rules, and local storage

Sprint 5 – Wanted List and Price Check

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 5	09/28 – 10/04	As a bibliophile, I want to see what a Wanted book costs and record how I got each Owned book, so that I can plan purchases and remember gifts.	1. Verify that a Wanted book shows a current bookstore price. 2. Verify that moving to Owned asks whether the book was purchased or gifted.

Team Member	Task
Grace Hechavarria	Bookstore price lookup for the Wanted list; acquisition rules

Sprint 6 – Polish, Tests, and Midterm Documentation

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 6	10/05 – 10/11	As a reviewer, I want a tested, documented midterm build, so that the midterm review can focus on the design.	1. Verify that every tier has automated tests. 2. Verify that the README and the design document match the code.

Team Member	Task
Grace Hechavarria	Interface polish, automated tests, README, ADRs, and the design document

Sprint 7 – Midterm Peer Review

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 7	10/12 – 10/25	As a reviewer, I want to run the app from the README and have my findings addressed, so that the review leads to improvements.	1. Verify that the partner runs the app from the README alone. 2. Verify that every midterm finding is resolved or answered.

Team Member	Task
Grace Hechavarria	Address the midterm review findings
Yousef Habouh	File the structured midterm review

Sprint 8 – Device Testing and Hardening

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 8	10/26 – 11/08	As a bibliophile, I want the app to be dependable on my own phone, so that I can catalog my whole library.	1. Verify that every flow works on a physical iPhone or Android device in Expo Go. 2. Verify that the documentation matches the final code.

Team Member	Task
Grace Hechavarria	Test on physical devices; fix defects; update the documentation

Sprint 9 – Final Peer Review and Demo

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 9	11/09 – 11/22	As the instructor, I want a final review and demo, so that I can evaluate the finished design.	1. Verify that the final peer review is filed and addressed. 2. Verify that the SDD is updated to its final version.

Team Member	Task
Grace Hechavarria	Final fixes, demo, and the final SDD revision
Yousef Habouh	File the structured final review

5.2.2 Actual Timeline

Sprint 1 – Project Proposal and Approval

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 1	08/24 – 08/30	As the instructor, I want a clear project idea and platform, so that I can approve the scope.	1. Verify that the repository exists with a README. 2. Verify that the approval paragraph names the platform, features, and the one external integration.

Team Member	Task
Grace Hechavarria	Created the repository; drafted three project ideas; wrote and updated the approval paragraph (PR #1)

Sprint 2 – Tier Breakdown

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 2	08/31 – 09/06	As a developer, I want each tier's responsibilities written down, so that every class has an obvious home.	1. Verify that the README has a Presentation / Service / Domain / Data table. 2. Verify that each row names the system's real responsibilities, not generic ones.

Team Member	Task
Grace Hechavarria	Wrote the tier responsibilities table (PR #2)

Sprint 3 – C4 and UML Models

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 3	09/07 – 09/20	As a reviewer, I want C4 and class diagrams, so that I can understand the system before reading code.	1. Verify that the README has C4 Context, Container, and Component diagrams. 2. Verify that a UML class diagram shows the domain rules. 3. Verify that the diagrams render on GitHub (Mermaid).

Team Member	Task
Grace Hechavarria	Drew the C4 Level 1–3 diagrams and the UML class diagram in Mermaid

Sprint 4 – Initial Architecture and Core Logic (Issue #3)

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 4	09/21 – 09/27	As a bibliophile, I want to scan or type an ISBN and see the book's details, so that I can add it without typing.	1. Verify that the camera reads an ISBN barcode. 2. Verify that a typed ISBN works when there is no camera. 3. Verify that the lookup shows title, author, and cover, and saves nothing.
		As a bibliophile, I want a book I already own to count as another copy, so that my catalog has no duplicate entries.	1. Verify that scanning a cataloged ISBN increases the copy count. 2. Verify that no second entry is created.
		As a bibliophile, I want separate Owned and Wanted shelves and a To-be-read list, so that I know what to read and what to buy.	1. Verify that a book is either Owned or Wanted. 2. Verify that TBR lists Owned books, split Unread / Read. 3. Verify that the library survives an app restart.

Team Member	Task
Grace Hechavarria	Built the four-tier skeleton, the domain rules, the scan flow (camera and typed ISBN), the Open Library adapter, and the SQLite and JSON repositories (PR #4)

Sprint 5 – Acquisition Rules (Issue #5)

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 5	09/28 – 10/04	As a bibliophile, I want every Owned book to record whether I bought it or received it as a gift, so that I remember where each book came from.	1. Verify that a Wanted book cannot move to Owned without a purchase or a gift. 2. Verify that a gift requires the giver's name. 3. Verify that scanning straight into Owned asks the same question. 4. Verify that a refused move saves nothing.

Team Member	Task
Grace Hechavarria	Enforced the acquisition rules in the domain model and every path into Owned, with tests (PR #6); the bookstore price check was dropped (ADR-004)

Sprint 6 – Exact Edition, Class-Based Refactor, and Design System

Sprint	Dates	User Stories	Acceptance Criteria
Sprint 6	10/05 – 10/11	As a bibliophile, I want the app to find the exact edition I am holding, so that the publisher, binding, and cover match my copy.	1. Verify that Google Books and Open Library are both asked and merged. 2. Verify that every distinct cover is offered, plus "Your photo". 3. Verify that the binding is asked only when no source knows it.
		As a developer, I want the code built from classes with no duplication and imported stylesheets, so that it is modular and easy to review.	1. Verify that screen logic lives in a view model class, not in components. 2. Verify that no component defines a stylesheet or inline style. 3. Verify that the tier rules hold (architecture tests pass).
		As a bibliophile, I want a polished, consistent interface, so that the app is pleasant to use every day.	1. Verify that every screen uses the same tokens, font, and building blocks. 2. Verify that text meets WCAG AA contrast. 3. Verify that motion stops when Reduce Motion is on.

Team Member	Task
Grace Hechavarria	Added the Google Books adapter, merged and cached lookups, the cover picker and own-cover photo, and the binding question; refactored the presentation into an MVVM view model; built the design system; brought the suite to 88 automated tests; wrote ADRs 006–009 and this SDD

6 Design

6.1 Definition of Design

Design in software engineering converts requirements into a blueprint for the system: its architecture, its main elements, and how they interconnect. The design phase decides what the elements are and how they communicate before (and while) they are built and tested. Sommerville (2021) describes software design as the process of identifying system components and their relationships so that the system can be implemented and verified.

6.2 Design Process

The design followed the course sequence. First, the problem and constraints were written as an approval paragraph. Next, the responsibilities of each tier were listed. Then the system was modeled at increasing zoom with the C4 model (Context, Container, Component) and UML class and sequence diagrams, and each significant decision was captured in an ADR.

The design was centered on the core workflow, scanning a book:

1. The user scans or types an ISBN.
2. The system looks up the exact edition in its book sources.
3. The user confirms the edition and picks the cover that matches their copy.
4. The user chooses a shelf, how the book was acquired (for Owned), and whether it has been read.
5. The system applies the domain rules and saves the book.

The domain rules were modeled as behavior of the domain classes rather than checks in the screens, so they hold no matter which screen triggers them. After the midterm build worked, the code was refactored toward the object-oriented techniques required by the course (abstract base classes, interfaces, polymorphism, composition, and design patterns), and the diagrams were regenerated to match the code.

6.3 Graphical Representation of the Design

6.3.1 Use Case Diagram

Figure 18 shows what the bibliophile can do. Scanning *includes* confirming the edition and choosing a shelf; choosing Owned *includes* recording a purchase or gift. A duplicate ISBN *extends* scanning into adding a copy. The two book sources are external actors of the scan use case.

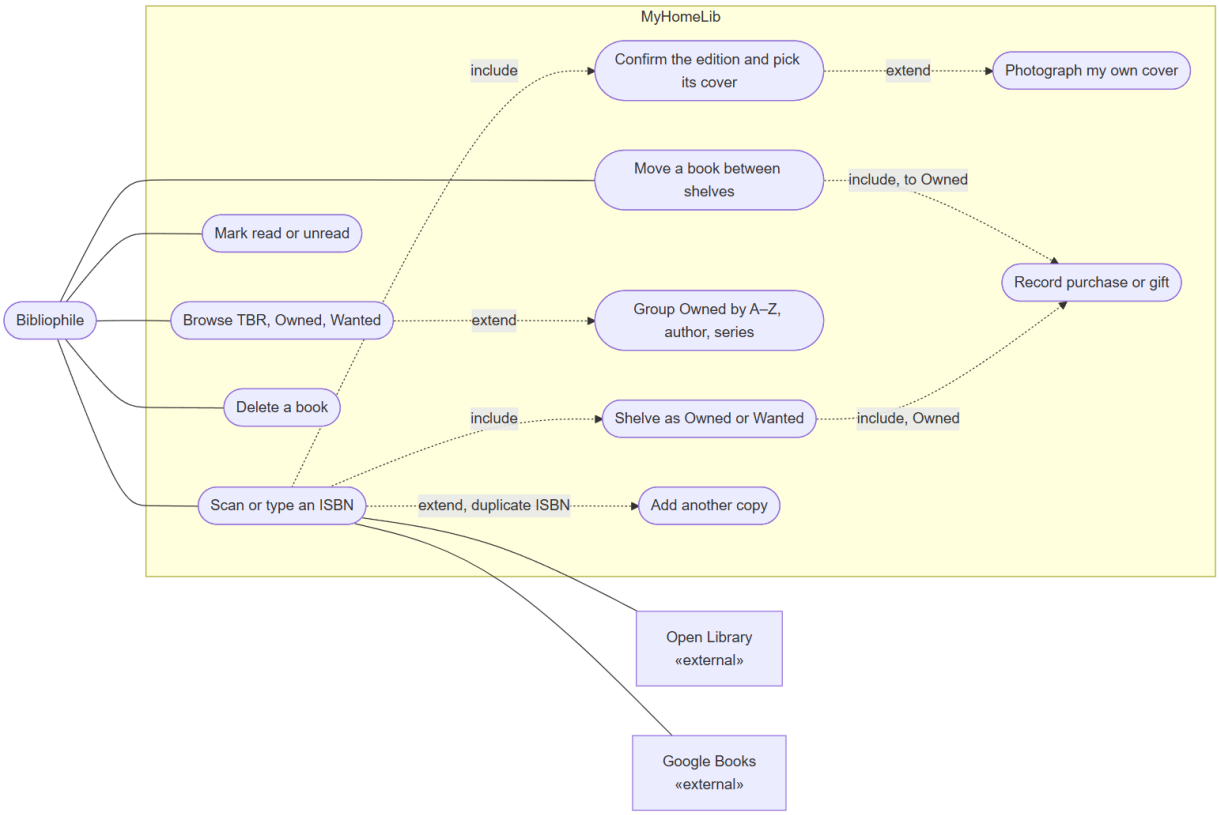


Figure 18: Use Case Diagram

6.3.2 Activity Diagram

Figure 19 shows every decision in the scan flow, including the typed-ISBN fallback, the unknown-ISBN exit, the duplicate-copy shortcut, the optional binding question, and the gift branch.

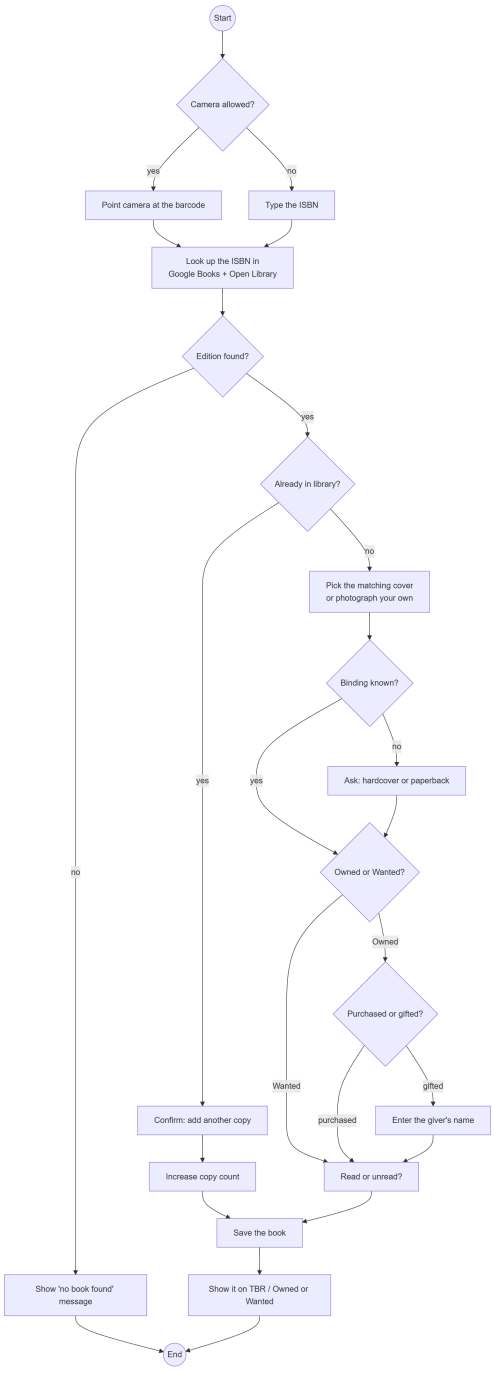


Figure 19: Activity Diagram: scanning a book

6.3.3 Class Diagram

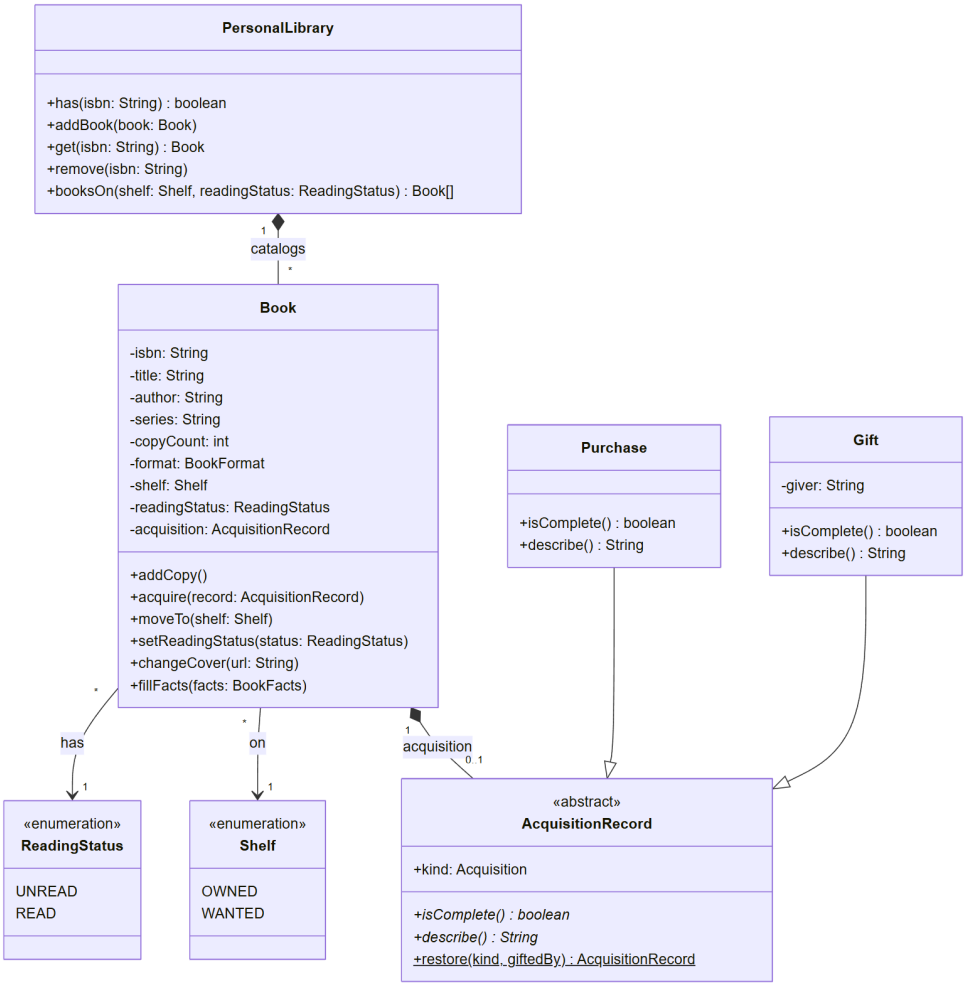


Figure 20: Class Diagram: Domain model

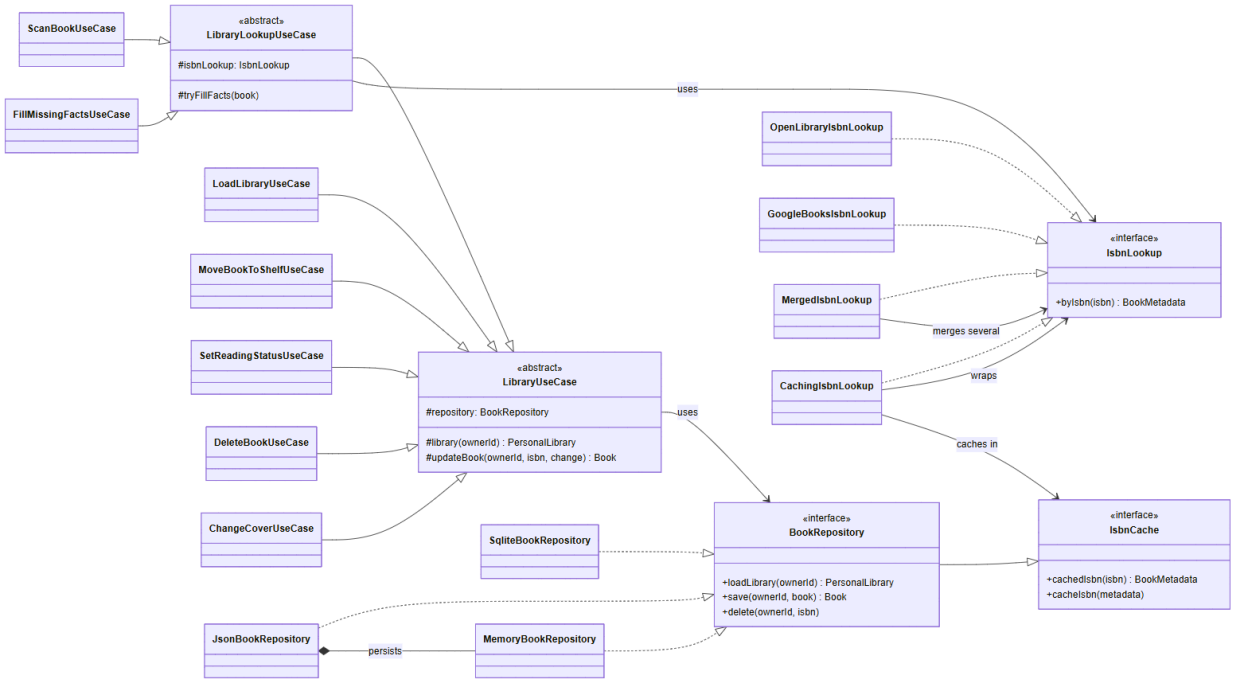


Figure 21: Class Diagram: Service use cases, ports, and Data adapters

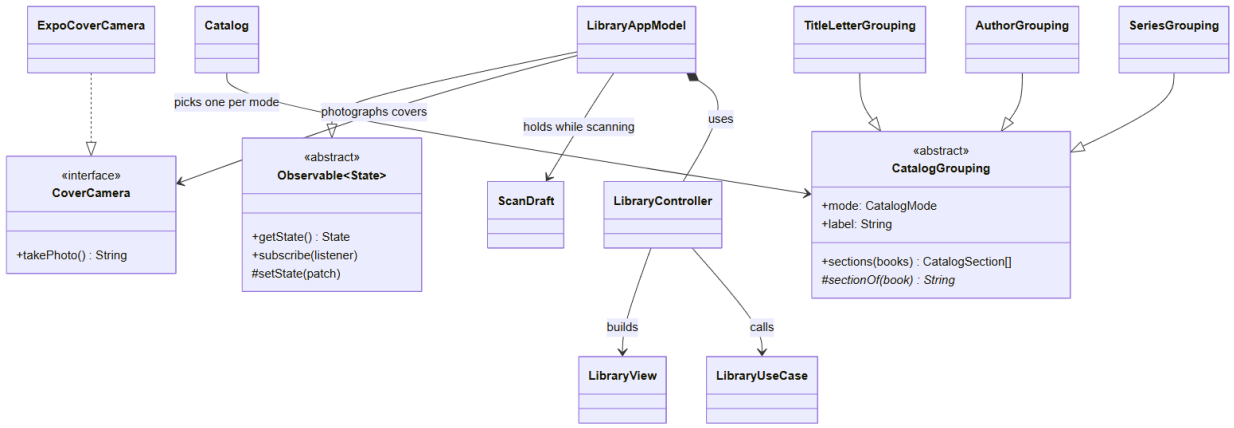


Figure 22: Class Diagram: Presentation view model and catalog strategies

Figures 20, 21, and 22 show the classes. The object-oriented techniques used, and where:

Technique	**Where it is used**
Encapsulation	`Book` keeps its state private; it changes only through rule-checking methods.
Abstract class + inheritance	`LibraryUseCase` → `LibraryLookupUseCase` → use cases; `AcquisitionRecord`; `CatalogGrouping`; `Observable`.
Polymorphism	`Purchase` and `Gift` answer `isComplete()` and `describe()` their own way.
Interfaces	`BookRepository`, `IsbnLookup`, `IsbnCache`, `CoverCamera`.

Technique	Where it is used
Composition	`PersonalLibrary` composes `Books`; `JsonBookRepository` composes `MemoryBookRepository`.
Template Method	`LibraryUseCase.updateBook` (load → change → save).
Strategy	`TitleLetterGrouping`, `AuthorGrouping`, `SeriesGrouping`.
Decorator	`CachingIsbnLookup` and `MergedIsbnLookup` wrap other lookups.
Factory	`UseCases` builds every use case; `AcquisitionRecord.restore`.
Data Mapper	`BookRowMapper` is the only row ↔ `Book` translation.
Observer / MVVM	`LibraryAppModel` extends `Observable`; React subscribes with `useObservable`.
Dependency injection	`CompositionRoot` passes adapters to the use cases; tests pass fakes.

6.3.4 Sequence Diagram

The sequence diagrams for scanning a book and for moving a Wanted book to Owned are shown in Figures 15 and 16 (Section 4).

6.4 Rationale Management

Rationale management records why each design decision was made. In this project every significant decision is recorded as an ADR in `docs/adr/`, so the reasoning stays next to the code and can be reviewed together with it.

ADR	Decision	Status
001	TypeScript + Expo Go modular monolith with four tier folders	Accepted (external-source line amended by 006)
002	SQLite on the device, JSON fallback on the web, one port	Accepted
003	Open Library for ISBN metadata (no key)	Accepted, amended by 006
004	No bookstore scraping	Accepted
005	Expo Go instead of an app-store binary	Accepted
006	Google Books as a second book source (optional key)	Accepted
007	MVVM presentation with an observable view model	Accepted
008	Design system: tokens, stylesheet imports, shared UI building blocks	Accepted
009	Ports and adapters between Service and Data	Accepted

6.4.1 Issues Addressed

The first issue was **portability**: the review partner had to run the app without a Mac or paid accounts, which ruled out native builds and led to Expo Go. The second issue was **trustworthy data**: a catalog is only useful if it cannot contain duplicate entries or Owned books with no record of how they were acquired, so those rules had to live in the domain model where no screen can bypass them. The third issue was **matching the physical copy**: early testing showed that one ISBN can map to a wrong or missing cover, so the lookup had to become edition-first, consult a second source, and let the owner choose. The fourth issue was **keeping the code reviewable**: logic that had grown inside one large screen component had to move into classes that can be tested and diagrammed.

6.4.2 Alternatives Considered

- **Native iOS app (Swift/Xcode)** or a **Flask web app**: rejected; the first needs a Mac, and the second is not a mobile app (ADR-001, ADR-005).
- **Scraping bookstore prices for the Wanted list**: rejected for reliability and terms-of-service reasons (ADR-004).
- **Open Library only**: simpler, but often wrong for a specific printing; Google Books was added as an optional second source (ADR-006).
- **Keeping screen logic in React hooks**: faster to write, but untestable without rendering and not class-based; replaced by an MVVM view model (ADR-007).
- **A UI component library**: would add dependencies and a generic look; a small token-based design system was built instead (ADR-008).
- **Plain JavaScript instead of TypeScript**: JavaScript is required by React Native, but it has no interfaces, abstract classes, or access modifiers, and it reports type mistakes only at run time. TypeScript was chosen because those features carry the course's object-oriented design straight into the code (the UML diagrams match the classes), the compiler refuses an adapter that does not implement its port, and all four tiers and the tests share one language.
- **Microservices or a backend server**: rejected; the library is private to one user and must work offline on a phone, so a server would add cost and failure points without benefit. A cloud store could later be added as one more `BookRepository` adapter.

6.4.3 Decisions Made

The design is centered on the scan workflow and the collection rules. The rules are methods of the domain classes; the use cases only orchestrate load, change, and save; the Data tier is replaceable behind two ports; and the screens are thin views of one observable view model. External sources are combined behind a single interface, so adding or removing a source does not change any caller.

6.4.4 Criteria Used to Guide Decisions

- The design must satisfy the course constraints (portable platform, one store, no scraping, documented architecture).
- The diagrams must match the code, and automated tests must keep it that way.
- Each class must have one responsibility, and no logic may be written twice.
- The design must stay simple enough to finish in one semester and explain in a review.

6.4.5 Design Debate and Final Resolution

The main debate was between keeping the external integrations to the minimum (Open Library only) and matching the user's exact copy. Feedback from the instructor favored limiting integrations and removing scraping, while testing showed that a single free source produced wrong covers for many editions. The resolution was to add Google Books only through its official API, make its key optional so the app still runs without it, merge both sources behind the existing `IsbnLookup` port, and always let the owner choose or photograph the cover. This kept the architecture unchanged, respected the no-scraping rule, and met the user's need. A second debate concerned how much structure to add for the course's object-oriented requirements; the resolution was to introduce each pattern only where it removed real duplication or made a rule testable, and to document each one in the table above.

7 Verification

Verification in software is defined by the IEEE 1012 standard as "the process of evaluating software to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase" (IEEE, 2024).

MyHomeLib was verified in two complementary ways. First, **automated tests** (Jest) exercise every tier: 88 tests in 7 files, including tests that walk the view model through each user flow without any user interface, and tests that enforce the architecture rules. Second, each user story was **walked through in the running application**, and the screenshots below show the results. The numbered cases match the Test-Case column of the RTM.

Test file	What it verifies	Tests
domain.test.ts	Book and library rules: copies, acquisition, shelves, reading status	17
useCases.test.ts	Every use case against in-memory adapters	23
bookSources.test.ts	Open Library and Google Books parsing, merging, failures, caching	16
appModel.test.ts	Every user flow through the view model, no UI	17
controller.test.ts	Controller, library view, JSON round trip	6
catalog.test.ts	A–Z, author, and series grouping	3
architecture.test.ts	Tier dependency rules and stylesheet rules	6
Total	**Test Suites: 7 passed; Tests: 88 passed**	**88**

Table 3: Automated test suite results (`npm test`, October 10, 2026)

7.1 Sprint 1 – Project Proposal and Approval

Process	Verification	Figure
(1) We open the repository's README. It contains the approval paragraph, which names the platform (Expo Go on iOS and Android, with Expo web as a fallback), the core features, and the external book source.	Verify that the approval paragraph names the platform, features, and integration	README

7.2 Sprint 2 – Tier Breakdown

Process	Verification	Figure
(2) We read the README's tier table. Each of Presentation, Service, Domain, and Data lists this system's own responsibilities, and each matches a top-level folder in the code.	Verify that each tier's real responsibilities are written down	Fig. 2

7.3 Sprint 3 – C4 and UML Models

Process	Verification	Figure
(3) We open the README on GitHub. The C4 Context, Container, and Component diagrams and the UML class and sequence diagrams render from Mermaid, and the class names match the code (the architecture tests keep the tiers as drawn).	Verify that the C4 and UML diagrams exist, render, and match the code	Fig. 1, Fig. 20

7.4 Sprint 4 – Scanning, Shelves, and Copies

Process	Verification	Figure
(4) We open the app on the To be read tab with a small library. Owned unread books are listed; the switch shows the unread and read counts.	Verify that TBR lists Owned books split Unread / Read	Fig. 23
(5) We switch to Read. Only finished Owned books appear, marked with a leaf. With an empty library the shelf explains itself and offers Scan.	Verify that reading status is independent of the shelf	Fig. 23
(6) We open Owned and switch between A–Z, Author, and Series. Each grouping lists the same books under index headings.	Verify the three catalog groupings	Fig. 11
(7) In Series, the Dune series appears as a fan of covers with a count badge of 2; tapping it lists both books.	Verify series grouping and expansion	Fig. 11
(8) We scan the ISBN of Matilda, which is already cataloged. The popup says it is already in the library; confirming adds a copy, and the banner reports “Copies: 2” without a second entry.	Verify that a duplicate ISBN increments the copy count	Fig. 24

Process	Verification	Figure
(9) Repository tests save a library through browser (JSON) storage, reload it, and get the same books back, including a gift restored as a `Gift`; a cache test looks up the same ISBN twice and the book sources are asked only once.	Verify that the library persists and lookups are cached	Table 3

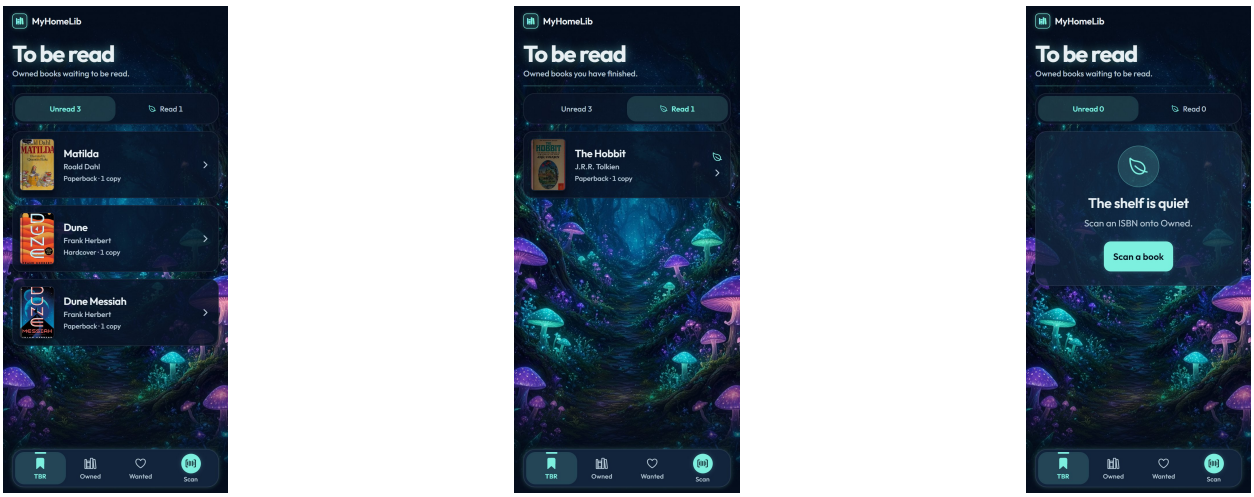


Figure 23: TBR unread, TBR read, and the empty state

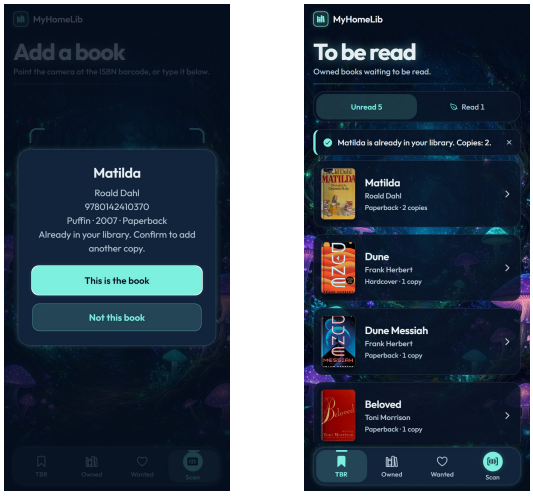


Figure 24: A duplicate ISBN offers to add a copy (left); the copy count becomes 2 (right)

7.5 Sprint 5 – Acquisition Rules

Process	Verification	Figure
(10) We open Beloved on the Wanted shelf and tap "To Owned". Before anything moves, the app asks whether it was purchased or gifted.	Verify that a Wanted book cannot move to Owned without a purchase or gift	Fig. 25
(11) We choose Gifted and tap Continue without a name. The popup stays open and nothing changes.	Verify that a gift requires the giver's name	Fig. 25
(12) We type the giver's name, continue, and answer Unread. The banner reports "Moved to Owned (unread)" and the book appears in Owned.	Verify that a complete gift moves the book	Fig. 26
(13) Domain tests try every move without an acquisition, with an empty giver, and with a blank giver; each is refused and the saved book stays Wanted.	Verify that a refused move saves nothing	Table 3
(14) While scanning a new book we choose Owned; the app asks "How did you get it?" before the reading question.	Verify that scanning into Owned asks for the acquisition	Fig. 28

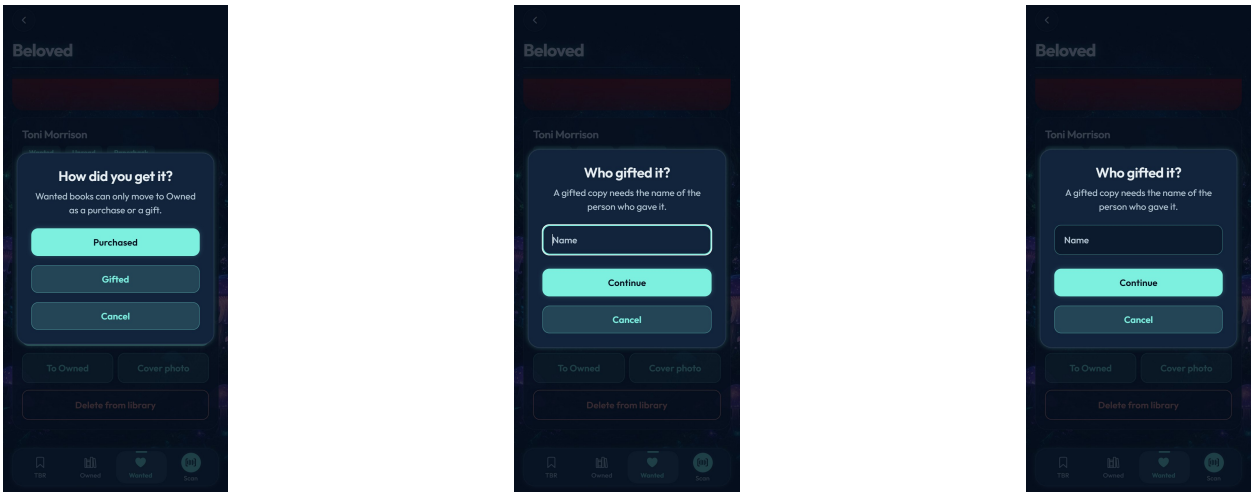


Figure 25: Moving to Owned asks for the acquisition (left); a gift needs the giver's name (center, right)

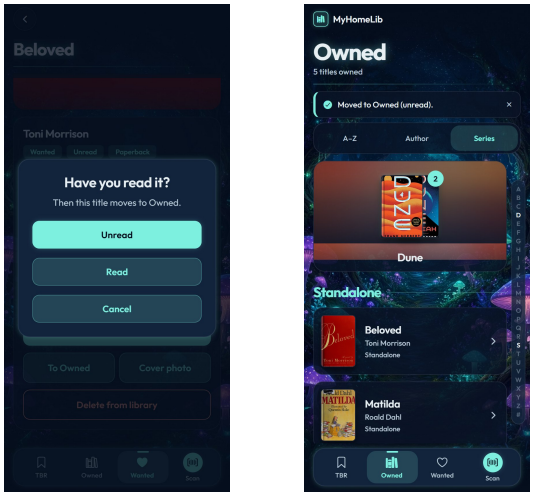


Figure 26: The last question is read or unread (left); the book moves to Owned (right)

7.6 Sprint 6 – Exact Edition, Refactor, and Design System

Process	Verification	Figure
(15) We open Scan. The camera preview shows the barcode target, and a typed-ISBN field is always available.	Verify the camera and the typed fallback	Fig. 27
(16) We type the ISBN of The Great Gatsby and look it up. The popup shows the edition (publisher, year) and saves nothing yet.	Verify that the lookup returns the edition and does not save	Fig. 27
(17) The popup offers every distinct cover the sources returned plus "Your photo". We tap the second cover; the check badge and the halo follow the choice.	Verify the cover picker	Fig. 27
(18) The sources did not state a binding, so the app asks "Hardcover or paperback?". A book whose binding is known skips this step (automated test).	Verify that the binding is asked only when unknown	Fig. 28
(19) We choose Owned, Purchased, and Unread. The banner reports "Added The Great Gatsby to Owned (unread)".	Verify that the scanned book is saved with every choice	Fig. 29

Process	Verification	Figure
(20) We start another scan and tap "Not this book". The banner reports "Nothing added" and the library is unchanged.	Verify that cancelling a popup changes nothing	Fig. 29
(21) We look up an ISBN that no source knows. The banner reports "No book found for ISBN . . . ".	Verify the unknown-ISBN message	Fig. 30
(22) Book-source tests feed recorded responses: edition data wins over work data, a different edition is never passed off as this one, every distinct cover is kept, and one source failing is tolerated.	Verify the edition-first merge	Table 3
(23) View-model tests photograph a cover with a fake camera and check that the photo is saved; a refused camera explains itself and changes nothing.	Verify "Your photo" and camera refusal	Table 3
(24) On a book's detail screen we tap "Mark read"; the banner reports "Marked read." and the book moves to the Read list.	Verify the read toggle	Fig. 30
(25) We tap "Delete from library"; a confirmation names the book and warns that every copy is removed. "Keep it" leaves it in place.	Verify that delete asks first	Fig. 30
(26) The architecture tests read every source file: tiers depend only downward, no component defines a stylesheet or inline style, and only `App.tsx` reaches the composition root. All 6 pass.	Verify the tier and stylesheet rules	Table 3
(27) We start the project with `npm start` and open it in Expo Go and in a web browser; no native build tools are required.	Verify the platform constraint	Fig. 10

Process	Verification	Figure
(28) Every text and background color pair was measured with the WCAG formula (lowest: 4.63:1, the danger button); cards announce “title, by author, …, read” to screen readers; list entrances stop when Reduce Motion is on.	Verify accessibility	–

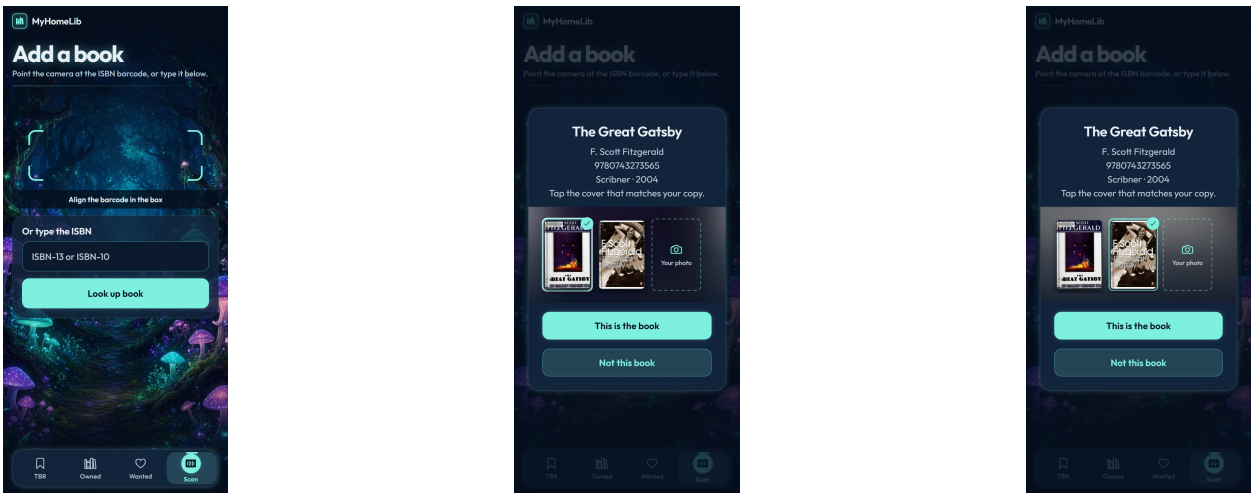


Figure 27: Scan screen (left); the edition with its covers (center); choosing the second cover (right)

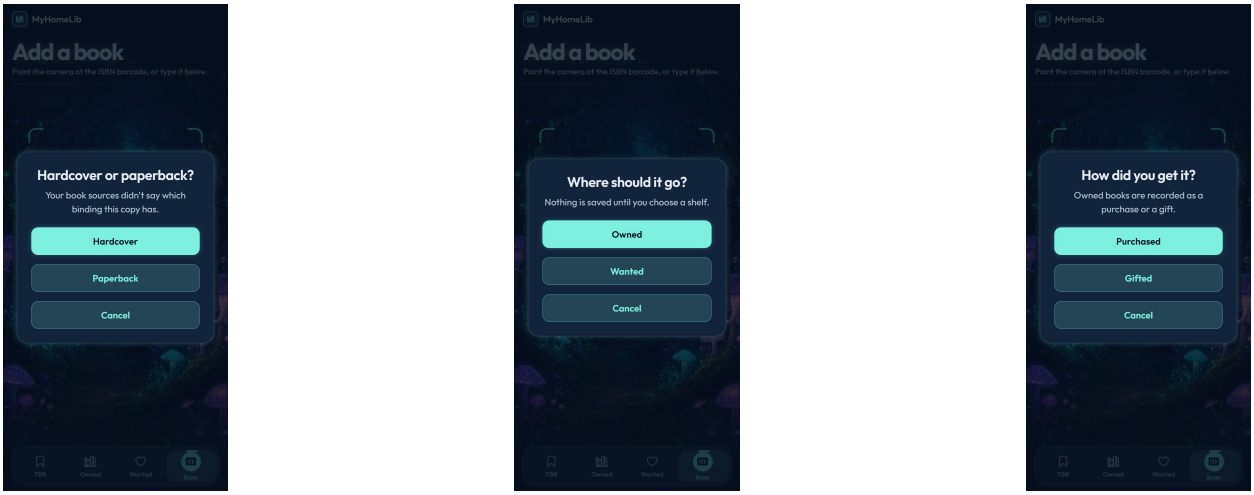


Figure 28: Binding question (left), shelf choice (center), and acquisition for Owned (right)

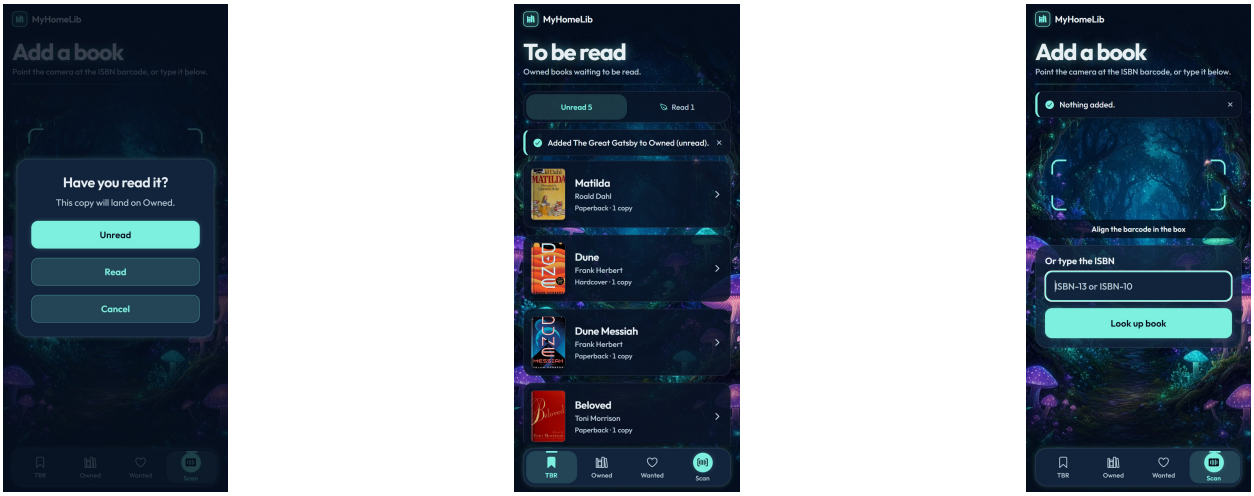


Figure 29: Reading question (left); the book is added (center); a cancelled scan adds nothing (right)

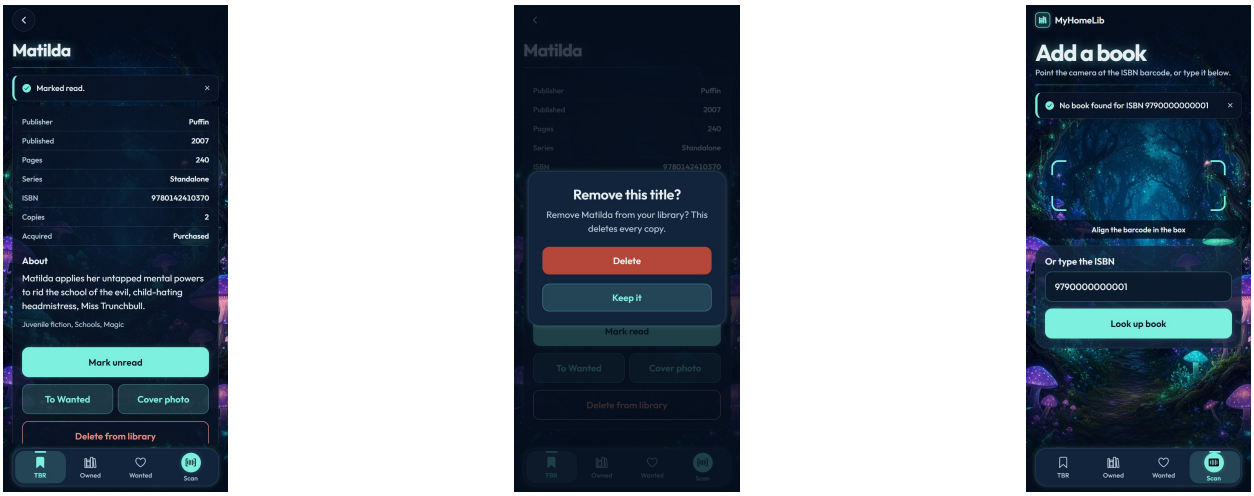


Figure 30: Marked read (left); delete confirmation (center); an unknown ISBN (right)

8 References

- Anthropic. (2026). *Claude* (Opus 5.5 model) [Large language model], accessed through *Claude Code* [Computer software]. Retrieved from https://www.anthropic.com/claude-code
- Brown, S. (n.d.). *The C4 model for visualising software architecture*. Retrieved from https://c4model.com/
- Expo. (n.d.). *Expo documentation*. Retrieved from https://docs.expo.dev/
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

- Google. (n.d.). *Books API*. Retrieved from https://developers.google.com/books
- Institute of Electrical and Electronics Engineers. (2024). *IEEE standard for system, software, and hardware verification and validation* (IEEE Std 1012-2024). IEEE.
- Internet Archive. (n.d.). *Open Library APIs*. Retrieved from https://openlibrary.org/developers/api
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Nygard, M. (2011). *Documenting architecture decisions*. Retrieved from https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions
- Sommerville, I. (2021). *Software engineering* (10th ed.). Pearson.
- World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. Retrieved from https://www.w3.org/TR/WCAG22/